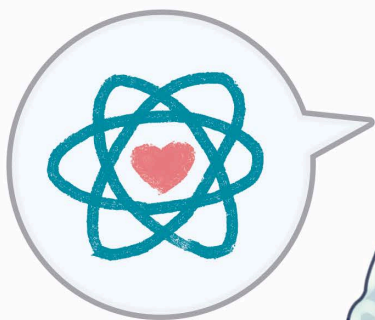


いあウト!



第5版
React 19

TypeScript
で始める
つらくない
React開発

大岡由佳

① 言語・環境編

関数型で型安全なReactプログラミング

仕事で使えるReact本は
これ! 現場の経験から生まれた書

シリーズ累計

3.5万部
突破!

読者
からの声

「会話形式で超わかりやすい!」
「この本でReactが怖くなくなりました」

BOOTH
技術書部門

1位

**りあクト！ TypeScript で始めるつらくない
React 開発 第5版 【① 言語・環境編】**

まえがき

この本は読者が**最新の React 19 の環境において、開発現場で即戦力になれる**ようにすることを目的として書かれたものです。従来の SPA¹に加え、Next.js などのフレームワークによる開発にも対応しています。

本書を執筆したきっかけは、フリーランスとして赴いた現場で React の未経験者に React を教えることの難しさを痛感した経験です。React は関数型プログラミングやコンポーネントベース・アーキテクチャ、テンプレートを使わない UI 記述など当時の Web 開発ではマイナーかつ難解なコンセプトの下に作られており、それをきちんと理解していない人の書いたコードをレビューするのはかなり大変でした。またベストプラクティスや組み合わせるライブラリ、開発に使うツールについてもトレンドの変遷が激しく、初手で筋の悪い技術を選んでしまっただけで苦勞している現場にもよく遭遇しました。

しかし当時はそういった React の難しさに正面から向き合った教材が見当たらず、ただ使い方を紹介しただけのものしかありませんでした。当時の彼らのように React がなかなか理解できない人たちに「**これ読んでおいて**」と**渡せるような教材**があれば多くの現場で役に立つだろうと考え、筆を執りました。

そうしてできた『りあクト!』シリーズは 2018 年に初版を発行してから改訂を重ねて本書で第 5 版となりますが、これまでにシリーズ累計で 3.5 万部以上が売れています。特にインターネットサービス企業で働く現場のエンジニアの方々からの評判が高く、各社で自主的に読書会が開かれたり、新人研修用の教材として採用されたりしています。

本書の構成

本書は「① 言語・環境編」「② React 基礎編」「③ React 実践編」の全 3 部からなっており、通して読むことで業務レベルの React 開発に最低限必要な知識がひととおり身につくようになっています。「これほどのボリュームで最低限なの?」という疑問はもっともですが、React は公開から 10 年以上も第一線で使い続けられているプロダクトで、数々の変遷を経ておりそのエコシステムまで含めて説明するとどれだけ書いてもカバーしきれません。ですので全体像をつかむことで「これをこう使えばこれができるんだな。じゃあ、あれが必要だからあの情報を調べよう」と**自走できるようになってもらえる内容と構成**にしたつもりです。

最新の ECMAScript による関数型プログラミングや TypeScript について知らない、自信のない方は

¹ Single-Page Application の略。最初に単一のドキュメントを読み込み、ブラウザの処理系を利用して動的に UI を更新するという Web アプリケーションの実装方法。アプリケーションのロジックはブラウザ上で実行され、多くの場合サーバの役割は静的ファイルの配信に留まる。

「① 言語・環境編」から。そのあたりはすでにマスターしているけど React は初めてという方は「② React 基礎編」から。React 経験者で最新の React 19 の新機能について知りたい方は「③ React 実践編」だけを読んでもいいでしょう。ただ React が Next.js のようなメタフレームワークで稼働するようになった経緯や、ここ数年で激変したツールチェーンの最新状況について興味のある方には「② React 基礎編」から読むことをおすすめします。

なお本書は本文すべてがシニアエンジニアと新人エンジニアの対話形式で記述されています。導入部分が対話形式のものはたまに見かけますが、すべてが小説仕立ての会話文になっているのは技術書ではかなりめずらしい存在のようです。このスタイルを採用した理由は **React に対して初学者が抱きがちな「Why?」** に丁寧に答えるため。ひょっとしたら筆者だけかもしれませんが、通常の解説調の文体では自分でも深く理解していない知識を表面的に書いてしまいがちです。だからまだ初心者だったころの自分を人格化して、浮かんだ疑問をそのつと付度なくぶつけるようにした結果、この形に落ち着きました。筆者自身が完全に腑に落ちるまで調べ尽くす必要があるので執筆にもものすごく時間がかかるのですが、初版から続くこのスタイルは読みやすくわかりやすいと好評を得ています。

本書の全3部を通してもっともマッチする読者像は作中の新人エンジニアがそうであるように、**業務での Web アプリケーション開発の経験がありながら React の経験は少ない職業エンジニアの方**です。プログラミングそのものが初心者の方には、本書はハードルが高くおすすめできません。読んで理解するには基本的な JavaScript や Web にまつわる知識が必要です。React の入門から実践までをカバーした本ではありますが、この本でプログラミングに入門しようとは思わないでください。なお、本書を読もうかどうか迷っている方に向けて、おそらく出てくるであろう疑問に以下でお答えします。

Q1. 「学習には公式ドキュメントで十分では？」

2023 年 3 月、公開 10 年めを迎えた React は公式ドキュメントをリニューアルしました²。旧ドキュメントと比べると情報が整理されており、ブラウザで直接試せるサンプルコードや洗練されたデザインもあっておおむね好評のようです。筆者もこれは、とにかく動作するアプリケーションを早く作れるようになりたいというニーズには合致していると感じます。

しかしこのドキュメントを読んで JSX の書き方はわかってても、JSX とは実際のところ何なのかとか、なぜ従来のような HTML テンプレートを使わないのかといったことはわかりません。JSX に限らずすべてがこの調子で、React の一見クセの強い設計や記法に対して、なぜそれを採用するに至ったかといった背景や思想についてはほとんど触れられていません。

思想や内部メカニズムの説明が不足しているため React の表面的な使い方は身についても、理解が

² <https://ja.react.dev/>

浅いため複雑なアプリケーションを作る場面でハマりやすい。たとえば再レンダリングの多発で動作が重くなったり無限ループが発生したりといった場合に出くわしても、仮想 DOM や Reconciler の動作原理を知らないとは根本的な原因を特定しづらいでしょう。

また React は UI を構築することに特化したライブラリ（自称）であるため、実際にアプリケーションを作るうえではルーティングライブラリやフォームライブラリ、UI コンポーネントライブラリなどさまざまなものを組み合わせる必要があります。公式ドキュメントにはそれらの選び方や使い方などは載っていないため、作れるようになるのは 1 ページ構成の素朴なアプリケーションだけです。それでは業務に耐えうるスキルは身につけませんよね。

さらに公式ドキュメントではほとんどのサンプルコードが JavaScript で記述されているのも問題です。現在の React 開発ではほとんどのアプリケーションが TypeScript で書かれています。State of React 2024 の調査では、TypeScript を全く使っていない開発者は全体の 8% しかいないという状況です。React アプリケーションを TypeScript によって型安全を保証しながら開発するというのは時代の趨勢であり、もはや当たり前のこととなっています。TypeScript で書かれたコードを JavaScript に書き換えるのは簡単ですが、その逆はハードルが高い。初学者は最初の段階から TypeScript で React の使い方を学ぶべきで、本書もそのような方針を取っています。

Q2. 「AI に書かせるので、React の歴史や思想なんて知らなくてよくない？」

昨今の ChatGPT を始めとした生成 AI の進歩には目を見張るものがありますね。筆者も本書執筆においてサンプルコードの作成、各種技術の調査、原稿の校正に AI を利用しました。開発での AI 支援コーディングも実際に行っていますが、React 開発に利用するうえでの危うさも少なからず感じています。

まず React のエコシステムは変化が早く、ベストプラクティスやデファクトスタンダードが数年で入れ替わることがめずらしくありません。そのため AI が古い API や非推奨の書き方を提案してしまうことがままあります。また React およびその周辺技術はアップデートが頻繁で、それに伴って普及を待つことなくすぐにでも活用したくなる機能も多い。しかしリリースされたばかりの機能は学習ソースが少なく、指示しても AI がそれを使ってくれない可能性が高いです。

また、構文的には正しいが意図や文脈に対してズレているコードが生成されることもよくあります。たとえば状態管理が必要ない場面で Redux を使ったコードを提案してきたり、副作用を乱用して変な挙動を引き起こしたり、ネイティブフォームで済む場面で重量級のフォームライブラリを持ち込もうとしたり。またサーバで必要なバリデーションを行わずセキュリティホールを作り込んだり、クライアントに公開してはいけない情報を埋め込んでしまうようなリスクもあるでしょう。

しかし表面上は動作することが多いので、これらの問題を読み解く力がないとそのまま採用してしまい、技術的負債が満載でメンテナンス性の悪いコードが量産されてしまいます。そのような眼を養うには歴史や思想、内部のメカニズムなどからの React への深い理解が必要だと筆者は考えます。AI

に質の高いコードを書かせる指示を出せるようになるため、AI が書いたコードを正しくレビューできるようにするためにも、広く知識を身につけあなた自身の React への解像度を上げる必要があります。

Q3. 「Next.js のようなフレームワークを学ぶのって必要なの？」

React 19 で導入された新機能の目玉は、SPA ではなくサーバで稼働させることを前提としたものとなっています。これを利用するには React 単体では難しく、ほとんどの場合は Next.js や React Router (旧 Remix) といったメタフレームワークに組み込まれて運用することになります。くわしくは「② React 基礎編」の「10-2. 第4世代：メタフレームワークの時代」で説明していますが、そのようなケースは昨今急増しており、もはや新規プロジェクトで React を昔ながらの SPA で運用するのは社内向けツールなどニッチな領域に限られるのではないのでしょうか。

ただし React を学び始めるのにいきなりフレームワークを使うのがいい方法だと筆者は考えません。フレームワークはそれぞれの思惑をもって React 本体とは別のチームが開発しているものであり、それを理解したうえで選定するべきものだからです。ゆえに本書では React の基本的な部分は SPA の環境で学び、差異が生じる機能についてはそれに加えて Next.js と React Router の計3パターンについて学ぶという形を取ることにしました。2025年5月現在、市場のシェアは Next.js が圧倒的に大きいですが、本書ではできるだけ中立的な立場で2つのフレームワークを紹介しています。

今や実際の現場では、フレームワークを使うことは必須と言っていいでしょう。本書はフレームワークをくわしく紹介しながらもフレームワーク・アグノスティック³な、類を見ない React 解説書です。読者には本書を読んだうえで、ぜひご自身でどのフレームワークを使うべきか判断していただきたいです。

サンプルコードについて

本文で紹介しているサンプルコードは、GitHub に用意した下記の専用リポジトリにて、章・節ごとに分けてすべて公開してあります。

<https://github.com/klemiuary/Riakuto-StartingReact-ja5.0>

学習を効率的に行うためにも、本文を読み進めながらこれらのコードを実際に実行することを強くおすすめします。またコードをご自身で変更して挙動のちがいを確かめてみるのも理解の助けになるはずです。

³ agnostic とは一般には不可知論者のこと。コンピュータの文脈では、特定の技術に依存しない設計であることを指す。

なおリポジトリのコードと本文に記載されているコードはまったく同じではない場合があります。紙面に適切になるよう改行位置を調整したり、文脈に関係しない箇所を省略したりしているためです。ご注意ください。

本文および上記リポジトリに掲載しているコードは読者自身のプログラムやドキュメント、ブログ記事などに流用してかまいません。それにあたりコードの大部分を転載する場合を除き、筆者に許可を求める必要はありません。出典を明記する必要はありませんが、そうしていただければありがたいです。

本書について

登場人物の紹介

柴崎雪菜（しばさき ゆきな）

都内のとあるインターネットサービスを運営する会社のテックリード・エンジニア。2016 年ごろから React を使っている。本格的なフロントエンド開発チームを作るための中核的人材として、今の会社に転職してきた。チームメンバーを集めるため人材採用にも関わり自ら面接も行っていたが、彼女の要求水準の高さもあってなかなか採用に至らない。そこで「自分が React を教えるから他チームのエンジニアを回してほしい」と上層部に要望を出し、社内公募が行われた。

秋谷香苗（あきや かなえ）

柴崎と同じ会社に勤務する、新卒 2 年目のやる気あふれるエンジニア。入社以来もっぱら Ruby on Rails によるサーバサイドの開発に携わっていたが、柴崎のメンバー募集に志願してフロントエンド開発チームに参加した。そこで柴崎から「1 週間で戦力になって」といわれ、彼女にマンツーマンで教えるを受けることになる。

前版との差分および正誤表

過去の版からの変更点と、本文内の記述内容の誤りや誤植についての正誤表は以下のページに掲載しています。なお、電子書籍版では訂正したものを新バージョンとして随時配信していきます。

- 各版における内容の変更
<https://github.com/kleimiary/Riakuto-StartingReact-ja5.0/tree/main/CHANGELOG.md>
- 『リアクト！TypeScript で始めるつらくない React 開発第 5 版』正誤表
<https://github.com/kleimiary/Riakuto-StartingReact-ja5.0/tree/main/errata.md>

本書内で使用している主なソフトウェアのバージョン

• React (<code>react</code>)	19.1.0
• React DOM (<code>react-dom</code>)	19.1.0
• Vite (<code>vite</code>)	6.3.5
• TypeScript (<code>typescript</code>)	5.8.3
• Node.js (<code>node</code>)	18.20.8 / 22.15.0

目次

まえがき	2
本書について	7
登場人物の紹介	7
前版との差分および正誤表	7
本書内で使用している主なソフトウェアのバージョン	8
プロローグ	14
第1章 こんにちは React	16
1-1. 基本環境の構築	16
1-1-1. Node.js がなぜフロントエンド開発に必要なのか	16
1-1-2. Node.js をインストールする	19
1-1-3. 超絶推奨エディタ Visual Studio Code	24
1-2. React のプロジェクトを作成する	28
1-3. プロジェクトを管理するためのコマンドやスクリプト	36
1-3-1. pnpm	36
1-3-2. npm スクリプト	41
第2章 ライトでディープな JavaScript の世界	44
2-1. あらためて JavaScript ってどんな言語？	44
2-1-1. 世界でもっとも誤解されたプログラミング言語	44
2-1-2. 年々進化していく JavaScript	47
2-2. 変数の宣言	49
2-3. JavaScript のデータ型	53
2-3-1. JavaScript におけるプリミティブ型	53
2-3-2. プリミティブ値のリテラルとラッパーオブジェクト	55
2-3-3. オブジェクト型とそのリテラル	58
2-4. 関数の定義	61
2-4-1. 関数宣言と関数式	61
2-4-2. アロー関数式と無名関数	65
2-4-3. さまざまな引数の表現	68
2-5. クラスを表現する	70
2-5-1. クラスのようでクラスでない、JavaScript のクラス構文	70
2-5-2. プロトタイプベースのオブジェクト指向とは	72
2-6. 配列やオブジェクトの便利な構文	77
2-6-1. 分割代入とスプレッド構文	77

2-6-2. オブジェクトのマージとコピー	82
2-7. 式と演算子で短く書く	85
2-7-1. ショートサーキット評価	85
2-7-2. Nullish Coalescing と Optional Chaining	87
2-8. モジュールを読み込む	89
2-8-1. JavaScript モジュール三國志	89
2-8-2. ES Modules でインポート／エクスポート	94
2-9. それでも知っておくべき this の挙動	97
第3章 関数型プログラミングでいこう	101
3-1. 関数型プログラミングは何がうれしい？	101
3-2. コレクションの反復処理	107
3-2-1. 配列の反復処理	107
3-2-2. オブジェクトの反復処理	113
3-3. JavaScript で本格関数型プログラミング	115
3-3-1. あらためて関数型プログラミングとは何か	115
3-3-2. 高階関数	117
3-3-3. カリー化と関数の部分適用	118
3-3-4. 閉じ込められたクロージャの秘密	120
3-4. 非同期処理と例外処理	125
3-4-1. Promise で非同期処理を扱う	125
3-4-2. async と await	128
3-4-3. 複数の Promise をまとめて扱う	130
3-4-4. JavaScript の例外処理	134
第4章 TypeScript で型をご安全に	137
4-1. 時代が望んだ言語 TypeScript	137
4-2. TypeScript の基本的な型	144
4-2-1. 型アノテーションと型推論	144
4-2-2. JavaScript と共通のデータ型	147
4-2-3. Enum 型とリテラル型	150
4-2-4. タプル型	153
4-2-5. any、unknown、never	154
4-3. 関数とクラスの型	157
4-3-1. 関数の型定義	157
4-3-2. TypeScript でのクラスの扱い	161
4-3-3. クラスの2つの顔	165
4-4. 型の名前と型合成	168
4-4-1. 型エイリアス VS インターフェース	168
4-4-2. ユニオン型とインターセクション型	171

4-4-3. 型の Null 安全性を保証する	174
4-5. さらに高度な型表現	177
4-5-1. 型表現に使われる演算子	177
4-5-2. 条件付き型とテンプレートリテラル型	180
4-5-3. 組み込みユーティリティ型	184
4-5-4. 関数のオーバーロード	188
4-6. 型アサーションと型ガード	191
4-6-1. as による型アサーション	191
4-6-2. 型ガードでスマートに型安全を保証する	193
4-7. モジュールのインポート／エクスポート	196
4-8. TypeScript の環境設定	201
4-8-1. コンパイラオプション strict	201
4-8-2. tsconfig.json の設定項目	204
4-8-3. 複数の tsconfig.json を連携させる	210

他巻目次

「② React 基礎編」目次

第5章 JSX でUI を表現する

- 5-1. なぜ React は JSX を使うのか
- 5-2. JSX 構文の書き方

第6章 React をめぐるモダン Web 開発の歴史（前編）

- 6-1. 第1世代：Ajax と DOM 操作のためのツールキット
- 6-2. 第2世代：SPA フレームワークの登場
- 6-3. パラダイムシフトを起こした React

第7章 ビルドツールでプロジェクトを運用する

- 7-1. React 開発におけるビルドツールの変遷
- 7-2. Vite で開発環境を構築する

第8章 コンポーネントの基本を学ぶ

- 8-1. UI コンポーネントライブラリの導入
- 8-2. コンポーネントに Props を受け渡す
- 8-3. コンポーネントに状態を持たせる
- 8-4. コンポーネントで副作用を扱う
- 8-5. React におけるコンポーネント構文の変遷
- 8-6. コンポーネントからロジックを分離し、再利用できるようにする

第9章 リンターとフォーマッターでコード美人に

- 9-1. リンターでコードの書き方を矯正する
- 9-2. フォーマッターでコードを整形する
- 9-3. Git Hooks と連携して実行させる

第10章 React をめぐるモダン Web 開発の歴史（後編）

- 10-1. 第3世代：SPA モデル黄金時代
- 10-2. 第4世代：メタフレームワークの時代

「③ React 実践編」 目次

第 11 章 React のページをルーティングする

- 11-1. React におけるルーティングとは
- 11-2. React 用のルーティングライブラリ
- 11-3. React Router を使う
- 11-4. Next.js で App Router を使う

第 12 章 外部リソースのデータを取得する

- 12-1. 各種の外部データ取得手法
- 12-2. Fetch-on-Render アプローチ
- 12-3. Fetch-then-Render アプローチ
- 12-4. Suspense for Data Fetching

第 13 章 エラーをハンドリングする

- 13-1. Error Boundary とは
- 13-2. react-error-boundary を使う
- 13-3. Next.js の App Router でエラーハンドリング
- 13-4. React Router (フレームワーク) でエラーハンドリング

第 14 章 メモ化でレンダリングを効率化する

- 14-1. 計算結果をメモ化する
- 14-2. 関数定義をメモ化する
- 14-3. React Compiler を導入する

第 15 章 フォームでデータを更新する

- 15-1. React でフォームの UI を実装するには
- 15-2. フォームを Action でハンドリングする
- 15-3. フォームライブラリを利用する

プロローグ

「おはようございます。本日からお世話になります、秋谷香苗です！」

「はい聞いてますよ、秋谷さんね。こちらこそよろしく。私はこのフロントエンド開発チームの……
といっても今のところは私と秋谷さんの2人だけなんですけど、リーダーの柴崎雪菜です」

「柴崎さんのこと私、前から知ってましたよ。柴崎さん、ウチに転職されてきて間もないけど凄腕の女性エンジニアって社内でもウワサになってましたし。今回、もちろん前から React に興味があったのもあるんですけど、柴崎さんに教わっていっしょに働けるチャンスだっていうので、この社内公募に手を上げたんです！」

「……そ、そう。ありがとう。やる気は十分ってことね。じゃあ今から、私が秋谷さんに何を期待しているとか、今後やってもらう予定のことを説明していこうと思うけど、いいですか？」

「はい、よろしくお願いします！」

「そうだ、その前に秋谷さんはいま入社何年めだったけ？ あと持ってるスキルについても教えてくれるかな」

「新卒で入社して今年で2年目です。入社してから1年ちょっと Ruby on Rails での Web アプリ開発に携わってました。使える言語は Ruby と、それに少しだけ JavaScript を。Rails アプリのフロントエンドにちょっとした効果を加えるくらいですけど。あと、業務では使ったことはないですが、入社前には Java を学んでました」

「なるほど、わかりました。React については？」

「興味があったので自分で勉強しようとしたんですけど、途中で難しくて挫折しちゃいました。Rails と考え方が全然ちがっていて、テンプレートとロジックを混ぜて書く感じなのも違和感があって、よくわからなかったんですよ。すみません……」

「いや、謝ることはないけれども。そうだね、秋谷さんのようにサーバサイドでよくある MVC⁴ フレームワークになじんでる人は、頭を切り替えることができないとそれが学習の妨げになりかねない」

「……そうなんですか？」

「React のアプリケーション設計思想はそもそも MVC とパラダイムが異なるので、その思想を理解しないまま飛び込んでもなかなか身につかないんだよね。でもそれらは随時、説明していくので心配しないで」

「はい、お願いします！」

「あはは、いい返事だね。で、まず何をやってもらうかだけど。今日から私がマンツーマンでつい

⁴ Model-View-Controller。UI を持つアプリケーションソフトウェアを実装するためのデザインパターンで、システムを機能別に Model、View、Controller の3つの要素に分割して設計する。Ruby on Rails を始めとする多くの Web アプリケーションフレームワークで採用されている。

て、秋谷さんに React 開発の基本を叩き込みます。そうだね、1 週間ほどで私とペアプログラミングで開発に参加してもらえるレベルになってもらいたいかな」

「ええっ、たったの 1 週間ですか！？ 無理無理、実質 5 日間しかないじゃないですか！？」

「いや、それだけあれば十分でしょう。私、教えるのうまいので」

「.....うーん、不安だなあ」

「だいじょうぶ、Rails は使いこなしてたんでしょう？ なら原理さえ理解できれば、そこまで React は難しくないから」

「わかりました！ 柴崎さんがそう言われるなら、覚悟を決めてがんばります！！」

第 1 章 こんにちは React

1-1. 基本環境の構築

1-1-1. Node.js がなぜフロントエンド開発に必要なのか

「ではまず基本的な環境の構築から始めたいんだけど、いちばん最初に入れる必要があるのが Node.js⁵ ね」

「あの一、Node.js ってよく聞くんですけど、それが何なのか実はよくわかってません……。最初に教えておいてもらっていいですか？」

「そうか、初心者だとそこからだね。ネットで『Node.js とは』って検索しても、まずそこで使われてる用語もわからないだろうし。JavaScript って本来はブラウザ上で動かすために作られた言語であって、そのままだとブラウザでしか動かないのは秋谷さんも知ってるよね？」

「はい、それはわかります」

「端的に言えば Node.js とは、JavaScript を Ruby や Python などと同じように秋谷さんの PC のターミナル上で実行するための環境を提供してくれるもののね。このようなソフトウェアをランタイムという。Node.js にはコアの言語処理エンジンとして Google Chrome 用に作られた V8⁶ を組み込んで、そこにローカルマシンで動かすためのファイルやネットワークの入出力機能とかが追加されている。だから Node.js を使えば、JavaScript を Ruby や Python のようにサーバサイド言語として動かすこともできるわけ」

「なるほど、わかりました。……。それでもまだ疑問があります。私たちがやろうとしているのはフロントエンド開発ですよ。だったらブラウザだけでしか動かないのって、別に困らなくないですか？」

「そうだねえ、じゃあブラウザだけで React を動かしてみようか。この HTML ファイルを Web ブラウザにドラッグ&ドロップして読み込ませてみて」

リスト 1: 01-env/script-react.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
```

⁵ <https://nodejs.org/ja/>

⁶ Google が開発するオープンソースの JavaScript エンジンであり、JIT コンパイル（ソフトウェア実行時にコードのコンパイルを行う）を介して動作する仮想マシンの形を取る。Google Chrome や Microsoft Edge といった Chromium ベースのブラウザ、Node.js や Deno などのサーバサイド JavaScript ランタイムに採用されている。 <https://v8.dev/>

```

<title>React Test</title>
</head>
<body>
  <div id="root"></div>
  <script type="module">
    import React from 'https://esm.sh/react@19';
    import { createRoot } from 'https://esm.sh/react-dom@19/client';

    const root = createRoot(document.getElementById('root'));
    root.render(React.createElement('h1', null, 'React works!'));
  </script>
</body>
</html>

```

「『React works!』って表示されました。やっぱり React もちゃんとブラウザだけで動くんじゃないですか」

「これは文法こそ新しくなっているけれども、Node.js 登場以前の原始的と言っていい手法だね。これで開発を続けていくと次のようなことに困ることになる」

- ローカルファイルを直接開くのは HTTP プロトコルで動作する本番環境と挙動が異なる (ex. セキュリティ制約や読み込みパスなど)
- 対応する JavaScript の仕様や使用可能な Web API が開発者の使っているブラウザに依存してしまう
- インポートするライブラリのバージョンやその依存関係を手動で管理しないといけない

「……うむむ、たしかにそれは問題ですね」

「今どきの Web 開発において、開発サーバとパッケージ管理システムはなくてはならないもの。Node.js はそれらが動作する基盤を提供してくれるわけ。それらを含めて Node.js を導入することで利用可能な、開発環境を快適にしてくれるツールを挙げてみよう。各ツールの概要や使い方については、必要になった段階で随時説明していくつもり」

- コードの変更を検知してブラウザのリロードなしにアプリケーションに反映してくれる高速な**開発サーバ**
- 依存関係を自動的に解決して設定ファイルで一元管理できる**パッケージ管理システム**

- 新しいバージョンの JavaScript や AltJS⁷ のコードを変換してあらゆるブラウザで動作できるようにしてくれる **コンパイラ**
- JavaScript や CSS ファイルを少数のファイルにまとめたり、不要な記述や重複を取り除いて軽量化してくれる **モジュールバンドラ**
- コードを解析して指定したルールに違反していないかチェックしてくれる **リンター**
- コードを自動整形してくれる **コードフォーマッター**
- ユニットテストや E2E テスト⁸を記述・実行するための **テストフレームワーク**

「Rails でもインストールすると `rails server` で起動する開発サーバがセットアップされてますし、パッケージ管理には `gem`⁹ や `Bundler`¹⁰ を使うので、たしかにこれらが無い開発は考えられないですね」

「なお Node.js のインストールパッケージには `npm`¹¹ というパッケージ管理ツールが同梱されているので、別途インストールの必要はないよ。これは RubyGems と Bundler を統合したようなツールで、依存関係も含めて管理してくれる」

「なるほど、Node.js が必要な理由がよくわかりました。丁寧な説明をありがとうございます！」

「よかった。じゃあこれから実際に Node.js をインストールしていこうか」

⁷ 「Alternative JavaScript」の略で、JavaScript を代替することを目的とした言語。可読性や保守性が高められており、一般的に JavaScript にコンパイルして使うことが想定されている。

⁸ 「End to End」テストの意。「端から端まで」の言葉通り、Web アプリケーションにおいては実際のユーザーが行うようにブラウザを操作して、期待通りの挙動となるかをシナリオに沿って確認するテストのこと。

⁹ 正確には「RubyGems」で、Ruby 標準のパッケージ管理システム。そのライブラリ群が「gem」と呼ばれる。またそのコマンドも `gem` である。<https://rubygems.org/>

¹⁰ `gem` 同士の依存関係とバージョンの整合性を管理してくれるツール。<https://bundler.io/>

¹¹ Node.js のために作られたパッケージ管理システム。パッケージをローカルにインストールしたり管理したりする機能に加え、リポジトリ機能も備えていて開発者が自身の開発したパッケージを `npm` で公開できる。2023 年の時点において提供するパッケージ数は 200 万以上と世界最大で、RubyGems や Maven Central (Java) など他言語のものを圧倒している。<https://www.npmjs.com/>

1-1-2. Node.js をインストールする

「Node.js のインストールのやりかたは色々あって Mac なら Homebrew¹²、Windows なら winget¹³ といったパッケージ管理ツールのリストに Node.js があるので、それらを使って入れるのがもっとも簡単で手っ取り早い。でも私たちはアプリケーション開発のプロなので、プロジェクトごとにそれぞれ異なるバージョンの環境を共存させることが必要になってくる」

「たしかにそうですね」

「だから開発者の間ではバージョンマネージャを使って Node.js をインストールするのが一般的。この分野で現在メジャーなのは古くからある **nvm**¹⁴ や **fnm**¹⁵、**Volta**¹⁶ あたりかな」

「Ruby でも同じようなものに **RVM**¹⁷ と **rbenv**¹⁸ がありますね」

「ただ、今挙げたようなツールは Node.js 単体でしか管理できない。どうせなら Ruby や Python といった他の言語ランタイムも同じツールで管理できると便利だよね。だから私は **mise**¹⁹ を使ってる。細かい説明は後回しにして、実際に mise で Node.js をインストールしてみよう。mise は Mac と Linux はもちろん、Windows まで広く対応してて、各環境でのインストール方法は公式ドキュメント通りにやれば OK。ここでは macOS 環境で Homebrew がすでに入っていて、シェルに zsh を使っている前提で進めていくよ」

```
$ brew install mise
$ echo 'eval "$(mise activate zsh)"' >> $HOME/.zshrc
$ echo 'export PATH="$HOME/.local/share/mise/shims:$PATH"' >> $HOME/.zshrc
$ exec $SHELL -l
$ mise use -g node
```

¹² Max Howell が中心となって開発している macOS 用のパッケージ管理システム。Debian 系 Linux に搭載されている APT のようにバイナリを配布するのではなく、都度ビルドを行う。ただしすべてのソースをビルドするのではなく、バイナリがあるものは **Bottle** というバイナリパッケージをインストールする。homebrew とは英語で自家醸造酒（ビール）を意味し、「ユーザーが自らパッケージをビルドして使用する」ことのメタファーとなっている。<https://brew.sh/>

¹³ Microsoft が開発しているパッケージ管理システム。正式名称は「Windows Package Manager」。コマンドラインからアプリケーションのインストールを管理できる。2021 年 5 月にバージョン 1.0 がリリースされた。

<https://learn.microsoft.com/ja-jp/windows/package-manager/winget/>

¹⁴ <https://github.com/nvm-sh/nvm>

¹⁵ <https://github.com/Schniz/fnm>

¹⁶ <https://volta.sh/>

¹⁷ <https://rvm.io/>

¹⁸ <https://github.com/rbenv/rbenv>

¹⁹ <https://mise.jdx.dev/>

「mise は他のツールとちがって環境変数 `PATH` 自体を動的に書き換えることでバージョンを管理しているので、公式の手順には上記の `export PATH=~` の記述はないんだけど²⁰、そのためインストールしたパッケージをターミナル以外の環境で実行しようとするが見つからないことがある。だからパスを通す記述を追加しておく。

はい、これで mise による Node.js のセットアップは完了。ターミナルから `node --version` を実行してみてください？」

「『v22.15.0』って表示されました。これは Node.js の最新版がインストールされてるってことですか？」

「そのとおり。なお OS やシェルなどの環境がちがう場合は、公式サイトを参照して適宜対応してね。なお mise は **プラグイン** というしくみを利用してさまざまな開発ツールやランタイムを扱ってる。Node.js を始めメジャーなランタイムのものはコアプラグインとして最初から組み込まれているので、別途プラグインをインストールする必要がないのね。mise `plugins --core` を実行するとどんなものがあるか確認できるよ」

「へー、Ruby や Python、Java に Go とかも対応してるんですね」

「さらにコミュニティのプラグインによって 700 以上のツールやランタイムに対応してるよ²¹。こういった複数の言語ランタイムに対応するバージョンマネージャは `anyenv`²² がその走りだったんだけど、2022 年で開発が止まってしまった。そしてその次にメジャーになったのが `asdf`²³。さらにその代替を目指している新進気鋭のツールが mise というわけ。asdf がシェルスクリプトベースなのに対し、mise はコンパイル型言語の Rust 製のために動作が機敏。また asdf はプラグイン周りの操作がけっこう煩雑なんだけど、mise はそれをかなり簡略化してくれてる。なお mise のプラグインは、asdf のものをそのまま流用する形になってる」

「ちなみにさっきから mise を『ミーズ』って発音してますけど、『マイズ』じゃないんですね。日本人だと『ミセ』って呼ぶじゃないですけど」

「mise の正式名称は『mise-en-place（ミーズ・オン・プラス）』。フランス語の言葉で、主に料理の文脈なんかで『下ごしらえ』の意味で用いられるそうだよ。mise のコマンドはたくさんあるんだけど、主なものを紹介しておこう。なお `[PLUGIN]` となっているところはプラグイン名の指定がオプションで、省略すると利用中のすべてのプラグインが対象になる」

²⁰ 「Getting Started | mise-en-place」 > 「3. Activate mise」
<https://mise.jdx.dev/getting-started.html#activate-mise>

²¹ <https://mise.jdx.dev/registry.html>

²² <https://anyenv.github.io/>

²³ <https://asdf-vm.com/>

プラグイン管理系

`mise plugins ls-remote` インストール可能なプラグイン（コアを除く）の一覧を表示

`mise plugins install <PLUGIN>` プラグインをインストール

`mise plugins update [PLUGIN]` インストール済みプラグインをアップデート

`mise plugins uninstall <PLUGIN>` インストール済みプラグインを削除

パッケージ管理系

`mise ls [PLUGIN]` プラグインパッケージのインストール済みバージョン一覧を表示

`mise ls-remote [PLUGIN]` プラグインパッケージのインストール可能なバージョン一覧を表示

`mise current [PLUGIN]` プラグインのアクティブなパッケージバージョンを表示

`mise outdated [PLUGIN]` プラグインの使用していないバージョンのパッケージを削除

`mise latest <PLUGIN>[@VERSION]` プラグインの利用可能な最新パッケージバージョンを表示。
`node@20` のように指定すると、**Node.js** の 20 系の最新バージョンが表示される

`mise install <PLUGIN>[@VERSION]` 任意のバージョンのパッケージをインストール。`node@latest` のように指定すると、**Node.js** の最新バージョンが対象になる

`mise uninstall <PLUGIN>[@VERSION]` 任意のバージョンのパッケージをアンインストール

`mise global <PLUGIN><@VERSION>` そのプラグインのグローバルに使うパッケージバージョンを設定

`mise local <PLUGIN><@VERSION>` そのプラグインのローカルに使うパッケージバージョンを設定

`mise prune [PLUGIN]` プラグインの使用していないバージョンのパッケージを削除する

`mise upgrade [PLUGIN]` プラグインのパッケージを最新版にアップグレードする。その際、旧バージョンのパッケージは削除される

「.....うーん、主なものだけでもこんなにあるんですか？ 複雑ですね」

「プラグインありきの `asdf` と互換性をできるだけ保ってるからね。他にもたとえば一連のサブコマンドで `ls` → `list` のようにエイリアスが張られていて、`asdf` のコマンドがほぼそのまま実行できるようになってる」

「ところで最初に使った `use` コマンドの説明がここにはありませんね」

「`use` は特殊なコマンドでね。たとえば `mise use -g php@latest` を実行したとする。それは次の3つのコマンドを順次実行したのと同じ結果になるの」

```
mise plugins install php
mise install php@latest
mise global php@latest
```

「へーえ。プラグインとその実際のバージョンのパッケージのインストール、それからアクティベートまで一気通貫でやってくれるのが `use` コマンドなんですね。ところすでにインストール済みの場合に同じコマンドを実行したらどうなるんですか？」

「`@latest` を指定している場合、最新版が更新されていたらそのバージョンのパッケージが新しくインストールされる。ただし最新版へのアップグレードなら `upgrade` コマンドのほうが便利だよ。同時に旧バージョンを削除してくれるからね。私は `mise outdated` で更新状況をチェックしたうえで、個別に `mise upgrade <PLUGIN>` でアップグレードすることが多いかな。それも面倒な場合は、`mise upgrade` 一発でインストール済みのパッケージをまとめて最新版にしてくれるよ」

「なるほど、`asdf` のプラグインシステムを踏襲しながら、できるだけプラグインの存在を意識せずに少ないコマンドで扱えるようになってるんですね。ところで話は戻りますが、`use` コマンドは `-g` オプションをつけずに実行するとどうなるんですか？」

「`-g` または `--global` オプションをつけないと、インストールはされるけどアクティベートはそのディレクトリ配下に限られる。たとえば Rust が一切インストールされてない状態で `~/Projects/MyRustApp/` で `mise use rust@1.70` を実行したとする。そのディレクトリ配下では `rust` コマンドで Rust ランタイムのバージョン 1.70.x が使えるけど、そこ以外ではそのバージョンはおろか `rust` コマンド自体が見つからなくなる」

「なるほど。それでその設定はどこに保存されてるんですか？」

「`use` を実行したディレクトリに、`mise.toml` というファイルが作成されて、そこで `mise` のローカル設定が管理されることになる。なおこれをリポジトリで共有しておくと、プラグインを指定せずに `mise install` を実行することで現状足りてないパッケージをまとめてインストールしてくれるよ。

しかも `mise` は設定ファイルとして `asdf` の `.tool-versions` はもちろん、`nvm` の `.nvmrc` や `nodenv` の `.node-version` にも対応しているので、チームがそれらのバージョンマネージャを使っている自分だけ `mise` に移行するなんてこともできる」

「それは乗り換えさせる気まんまんな仕様ですね……。ちなみに `-g` オプションを指定して `use` した場合の設定ファイルはどうなってるんでしょう？」

「グローバルな設定は、デフォルトでは `~/.config/mise/config.toml` に保存される。ただしローカルのときのように、設定ファイルの内容に沿った一括インストールはできないので、それはおぼえておいて」

「もうひとつ質問です。use する前に『mise ではこのツールも管理できるのかな?』と疑問に思ったときはどうしたらいいですか? たとえば Homebrew なら `brew search` ってコマンドがありますよね?」

「残念ながら mise にはプラグイン検索のためのコマンドはないの。だから `mise plugins ls-remote | grep kotlin` のように一覧から `grep` で抽出するしかないね」

「あらら、そうなんです」

「mise の説明はいったんここまでにして、次はインストールした Node.js を使ってみよう。すでに `node` コマンドが使用可能になってたはず。node コマンドのもっとも多い使い方は、JavaScript ファイルを用意して `node <JavaScript ファイル名>` を実行するもの。これでそのファイルのコードが JavaScript として評価され、上から順次実行される」

「そこは `ruby` コマンドと同じですね」

「そうだね。また、`node` とコマンド単体で実行すると REPL²⁴ が起動する。ちょっと実際にやってみようか」

```
$ node
> 4 * 8 + 1
33
> const foo = 'foo is string';
> foo
'foo is string'
```

「対話環境で JavaScript が実行できるんですね。Ruby でいう `irb` コマンドだ」

「そう、Node ではひとつのコマンドに統一されてるのね。最新バージョンではカーソルキー上下で入力履歴をたどれるのはもちろん、ある程度入力すると候補が表示されて Tab キーで補完できたり、変数の中身や式の結果は改行を入れなくても入力中に値が下に表示されたりと、けっこう高性能だよ」

```
> .help
.break      Sometimes you get stuck, this gets you out
.clear      Alias for .break
.editor      Enter editor mode
.exit        Exit the REPL
.help        Print this help message
.load        Load JS from a file into the REPL session
```

²⁴ 「Read-Eval-Print Loop」の略で、対話型の実行環境を意味する。キーボードから打ち込まれた命令を読み込み (Read)、評価・実行し (Eval)、結果を画面に表示し (Print)、また命令待ちの状態に戻る (Loop) ためこう呼ばれる。

```
.save      Save all evaluated commands in this REPL session to a file
```

Press Ctrl+C to abort current expression, Ctrl+D to exit the REPL

```
> .load 01-hello/01-env/rectangle.js
> const square = new Rectangle(5, 5);
> square.getArea();
25
```

「`.help` で REPL コマンドの一覧が表示される。ざっと見ておいてほしいけど、よく使うのは `.load` かな。指定したファイルの内容を現在のセッションに読み込んでくれるコマンドだね。node <JavaScript ファイル名> で直接実行するより、こっちのほうが色々挙動を試せて便利だから」

「へー、これよさそうですね。活用させていただきます」

「最後、ホームディレクトリに `.default-npm-packages` というファイルを作って次の内容を書き込んでくれる？」

リスト2: `.default-npm-packages`

```
typescript
tsx
typesync
```

「このファイルは何ですか？」

「Node の任意のバージョンをインストールしたとき、デフォルトでいっしょにインストールされる npm パッケージを登録しておけるの。Ruby なら `.default-gems`、Python なら `.default-python-packages` というファイルに相当する。ここに書いた各パッケージの内容については、おいおい説明していくから、今は流しておいて」

「わかりました」

「本来ならこのファイルは Node.js をインストールする前に用意しておきたかったんだけどね。ここに載せてるパッケージは後々必要になるので、以下のコマンドを実行してあらためてインストールしておこう」

```
$ xargs npm install -g < ~/.default-npm-packages
```

1-1-3. 超絶推奨エディタ Visual Studio Code

「開発環境の構築、次はエディタね。秋谷さんは今、開発にはどのエディタを使ってる？」

「VS Code ですね。チームには Vim 使ってる人がちらほらいましたけど、若手になるほど VS Code

第2章 ライトでディープな JavaScript の世界

2-1. あらためて JavaScript ってどんな言語？

2-1-1. 世界でもっとも誤解されたプログラミング言語

「とりあえず『Hello, World!』はできたけど React について本格的に学んでいく前に、その記述言語の知識をしっかり身につけておいてほしいの。Ruby on Rails を使いながら Ruby のことをあまり知らなかったり、Vue.js を使いながら JavaScript の知識がおろそかだったりする人がよくいるけど、React ではそれは通用しないので。

ウチのチームでは実際には TypeScript で開発していくわけなんだけど、TypeScript は JavaScript に静的型付けによる拡張された型システムを加えた上位互換の言語なので、まずはそのベースとなってる JavaScript から学んでいこう。といっても、秋谷さんは JavaScript での開発経験があるんだよね？」

「あっ、はい。Rails アプリの view 部分で、ちょっと凝った UI を実現するために JavaScript を書くことがあったので。ただちゃんと学んだことがあるわけじゃなく、その場その場で必要なところだけ調べながら書いてました」

「つまり条件分岐や繰り返し構文とかの基本的な文法は知ってて簡単な DOM 操作くらいはできるけど、それ以上に踏み込んだところまでの知識はないと。じゃあそのレベルでの感想でいいんだけど、秋谷さんは JavaScript にどんなイメージを持ってる？」

「うーん、基本的な構文は Java っぽいんだけど、要所要所でクセが強い印象がありますね。クラスもなんか簡易的だし、たまに出てくる『コールバック』っていうのよくわからないし……。周りの先輩たちも『できるだけ JS は使いたくない』って言ってました」

「……ああ、サーバサイドの Web アプリケーションフレームワークに長年慣れ親しんだ人たちの中には JavaScript のことを、HTML の飾り付けのためだけに使う書き捨て用の貧弱な言語だと思ってる人が今もいるみたいだからね。でもそれは、偏った価値観とアップデートされてない古い知識による不当な評価だと私は抗議したい。日本では JavaScript を悪く言う開発者は今でも多いけど、世界的にはずっと前から再評価されていて好きな言語として挙げる人も多い」

「そうなんですか？」

「うん。GitHub での利用状況や Stack Overflow で話題になった件数を集計してほぼ半年ごとに発表されてる Redmonk の言語ランキングでは、JavaScript は最初の 2012 年からほぼずっとトップにあり続けてて、その座を譲ったのはわずか 1 回しかない⁵⁰。

⁵⁰ 「RedMonk Top 20 Languages Over Time: June 2024」
<https://redmonk.com/rstphens/2024/09/12/top20-jun2024/>

使われてる数だけじゃ納得できないかもしれないだろうから、次は開発者による好き嫌いの調査を見てみよう。Stack Overflow による Developer Survey 2024⁵¹ での『Admired and Desired（称賛され、望まれている）』における言語のランキングでは、JavaScript は Python に次いで 2 位。回答者の 58.3% が称賛に票を入れている」

「へー、知りませんでした。世界的には JavaScript って、けっこう好かれてるんですね」

「JavaScript はたしかに初期設計が甘く、安全性を欠く挙動を生みかねない仕様を多く含んでる。でもそれらは近年のアップデートによってかなりカバーされてきてるし、進歩がめざましい静的解析ツールとの組み合わせで十分フォローできる。必要以上にそこをあげつらって JavaScript を使えない言語だと主張する記事を見かけるけど、それらはフェアな意見じゃないよ。それにいっぽうで JavaScript の基本的な設計の大枠は、あの時代にあって慧眼ともいえるものだったと思う」

「ふむふむ、話を聞いてると柴崎さんの JavaScript への評価ってかなり高いんですね。柴崎さんは JavaScript のどこをそんなに評価してるんですか？」

「そうだね。いくつか挙げるとしたらこんなところかな」

- ・ 第一級関数とクロージャをサポート
- ・ 構造体ともクラスインスタンスとも異なる、シンプルで柔軟なオブジェクト
- ・ 表現力の高いリテラル記法

「えーっと、ひとつめからして何のことやらさっぱりわからないんですけど……」

「JavaScript には秋谷さんのように Ruby や Java をメインにしている開発者にとってなじみのない概念が多くて、そこが彼らを困惑させて低い評価につながってる面もあるんだよね。これらを秋谷さんに今の段階で説明して、すぐ納得させてあげるのは無理かな。でもこれからステップを踏んで JavaScript を学んでいってもらうので、ひととおり終わるころにはこの意味が理解できるようになってるはずだよ」

「わかりました、じゃあ今、理解できそうな部分で質問させてください。そもそもの話なんですけど、JavaScript って Java とどういう関係なんですか？ 名前と構文が中途半端に似てますけど、実際のところはまったくの別物だし。だから私も最初、Java の簡易版のようなつもりで使おうとして混乱したんですよ」

「それにはちょっとした歴史的経緯があってね。生みの親である Brendan Eich は 1995 年ごろ当時

⁵¹ Stack Overflow が毎年実施している、有志の開発者を対象としたアンケートによる調査結果。2024 年は約 65,000 人からの回答があった。
<https://survey.stackoverflow.co/2024/technology#admired-and-desired>

Netscape の社員で、静的スコープと第一級関数をサポートする Scheme⁵² とプロトタイプベースのオブジェクト指向言語 Self⁵³ のいいとこどりをしてブラウザで動く言語を作ろうとした。それは最初 Mocha と名付けられ、その後 LiveScript と改名された。その過程で Netscape の経営陣は、この言語を普及させるには同じくブラウザで動作し、当時飛ぶ鳥を落とす勢いだった Java と構文を似させる必要があると判断した。そして彼にその改変を指示して、ついには言語の名前も JavaScript に変えさせたの⁵⁴」

「うーん。流行ってる言語に名前と構文を似せたら、その人気にあやかれるって発想がわかりませんが」

「でも結果的には大成功したわけだしね。ただそれが改名と構文改変のおかげだったのかは、今となってはわからない。名前については個人的に Mocha はアリだけど、LiveScript よりは JavaScript のほうがよかったかな。

とはいえ中身はまったく別のパラダイムで作られた言語なのに、名前と構文が似てるせいで秋谷さんのように混乱してしまう開発者が多いのも事実だね。JavaScript のことを『C の皮を被った Lisp』⁵⁵ だという人もいるけど、言い得て妙だと思う」

「ええ～、JavaScript って Java じゃなくて Lisp の仲間だったんですか？ つまりほとんど関数型言語ってこと？ どうりで関数周りの扱いが意味不明だと思ってました」

「基本的な構文が C ライクだから、たいていの開発者はわかってなくてもある程度書けてしまうんだよね。それでちょっと複雑なことをしようとする、なじみのない概念に突き当たって困惑する。

JavaScript を学ぶ上で、C や Java の延長にある言語という先入観はむしろ障害になるかもね」

「.....そうだったんですね」

「うん。Java と JavaScript は、マリオブラザーズとスーパーマリオくらいがうと思ってないと」

「すいません。その^{たと}え、全然ピンとこないんですが.....」

⁵² Lisp の方言のひとつ。関数を第一級オブジェクトとして扱うことができる（第一級関数）ほか、静的スコープ（「レキシカルスコープ」とも。構文構造のみから決定できるスコープのこと）が特徴。Scheme により Lisp 方言に静的スコープが広められた。

⁵³ クラスの硬直性を解決するべく生まれた「プロトタイプ」の概念に基づいたオブジェクト指向言語。JavaScript や Lua に概念的な影響を与えた。

⁵⁴ 「JavaScript: how it all began」

<https://2ality.com/2011/03/javascript-how-it-all-began.html>

⁵⁵ Douglas Crockford／水野貴明（訳）（2008）『JavaScript: The Good Parts —— 「よいパーツ」によるベストプラクティス』オライリー・ジャパン p.3

2-1-2. 年々進化していく JavaScript

「秋谷さんに言われて Ruby on Rails での JavaScript の使い方をちょっと見てみたけど、最近の Rails ではモダンな JavaScript がそのまま普通に書けるようになってるんだね」

「モダンな JavaScript？ JavaScript にモダンもクラシックもあるんですか？」

「なるほど、そこの区別からか。JavaScript の言語仕様にもバージョンがあってね、おおまかに ES2015 以降がモダン JavaScript と言われることが多いの」

「いーえすに一まるいちご？」

「正式名は『ECMAScript 2015』ね。JavaScript というのはさっきも言ったように元々 Netscape 社が開発していたプロダクトとしての名前で、Netscape 社が消滅してから二転三転して今は Oracle 社の登録商標になってる。その JavaScript が準拠している標準仕様の名前を **ECMAScript**（エクマスクリプト）という」

「へー。一般名より商品名のほうの認知度が高いつて、ホッチキスとかポストイットみたいですね」

「まあそんな感じだね。Ecma International⁵⁶ という IT 分野における国際的な標準化団体があるんだけど、そこで当時 Netscape 社が開発していた JavaScript を元に標準化が進められ、1997 年に ECMAScript として初版が公開された。ちなみに Ecma は他にも Microsoft オフィスが準拠してる Office Open XML とか C# や Dart なんかの仕様も決めてて、ECMAScript の仕様を策定しているのは **TC39** という委員会。

実は ECMAScript に準拠している言語は JavaScript だけでなく、Flash Player 上で動作するアプリケーションを開発するための ActionScript という言語はその仕様を拡張したものだった。Flash 自体がもうなくなっちゃったけどね」

「Flash、なくなっちゃいましたねえ……」

「そしてさっき言及した ES2015 というのは、ECMAScript 仕様の第 6 版で 2015 年に出版されたものね。ES5 が現在もっとも広く使われている版で、IE9 上でさえも動作する。だから多くの AltJS もコンパイラターゲットをこの ES5 にしてる」

「おるとじえーえす？」

「『Alternative JavaScript』の略で、JavaScript の代替言語のこと。JavaScript を使いやすくするための仕様を追加・変更したもので、最終的に JavaScript コードにコンパイルされる形式のことが多い。知ってるかどうかかわからないけど、昔の Rails は CoffeeScript⁵⁷ っていう JavaScript に Ruby と Python を足して 3 で割ったような言語がデフォルトで組み込まれてたんだよ」

⁵⁶ <https://www.ecma-international.org/>

⁵⁷ JavaScript をより簡潔に、読みやすくするために開発された言語。コンパイルにより JavaScript のコードに変換される。文法は Ruby や Python から影響を受けている。Ruby on Rails ではバージョン 3.1 以降 5.1 まですで、フロントエンドを記述するための言語として正式サポートされていた。
<https://coffeescript.org/>

「CoffeeScript って名前だけは聞いたことはありますね。使ったことはないですけど」

「ECMAScript は近年、ほぼ毎年のように新しい仕様が公表されてる。2025 年 4 月現在での最新版は ES2024、つまり ECMAScript 2024 で 2024 年の 6 月に正式仕様が公開された。ほぼ毎年、6 月には最新版が出るの」

「ECMAScript って年号をバージョン表記の代わりに使うんですね。Windows や Office みたい」

「2015 年に公開された第 6 版から、エディション名じゃなく年号付きの仕様書名で呼ぶことが推奨されるようになったのね。それまでは『ES5 (ECMAScript 5th Edition)』のようにバージョン番号で呼ばれてた。『ES6』のように表記している記事もないわけじゃないけど、それらはすべて ES2015 のこと。

なぜそこから呼び方が変わったのかというと、最初に言ったように ES2015 で大きな改変があってそこでモダンな仕様が一気に盛り込まれた記念すべき転換点の年だったから。ES5 までの JavaScript しか知らない人には、ES2015 以降の JavaScript は別の言語と思ってもらってもいいくらい」

「……そうなんですね、全然区別してなかったです」

「ただいっぽうで、JavaScript ほど後方互換性を大事にしてる言語は他にないともいえる。古いブラウザで動いてたプログラムが新しいブラウザで動かなくなるという状況を作らないために、基本的に後方互換性を破壊する変更は ECMAScript の改訂ではほとんど入らないのね。これはしょっちゅう後方互換性を破壊してる PHP や、バージョン 2 から 3 への移行が大騒動になった Python とかとはかなり対症的だよね。

ECMAScript の改訂仕様は古いものから年代を経た地層のように積み重なってる。だから古くて問題のある仕様も、掘り起こせば当時のままたくさん残ってるわけ」

「ふむふむ」

「でもそれらの『悪いパーツ』や『ひどいパーツ』⁵⁸は最新の開発ツールを併用すれば、うっかり使ってしまうようにうまく避けることができるので、実際にはそれほど気にする必要はないよ。そして最新の ECMAScript で追加されている仕様を駆使すれば、後発のモダンな言語たちにも引けを取らない快適で効率的な開発ができる」

「その『悪いパーツ』ばかり批判する人たちが見てる JavaScript と、柴崎さんたちが評価してるいいところ取りした最新の JavaScript は、同じ JavaScript といいながら実のところかなり別物なんだろうね」

「そうだと思うよ。ちなみに Vite で TypeScript テンプレートを指定してプロジェクトを作成すると、デフォルトでコンパイルに含められる ECMAScript のライブラリが ESNext、つまりそのバージョンの TypeScript がサポートするもっとも新しいバージョンの ECMAScript になってる」

「日々、追随していつてるんですね」

「それらを踏まえた上で、ここからは JavaScript の基本的な構文は押さえているものとして React と

⁵⁸ 『JavaScript: The Good Parts —— 「よいパーツ」 によるベストプラクティス』 p.117

TypeScript によるモダンフロントエンド開発で必要な部分にしばって JavaScript を説明していくよ」
「はい、お願いします！」

2-2. 変数の宣言

「ではまず変数の宣言から。ひと昔前のサンプルコードではカジュアルに `var` キーワードが使われてたりするけど、これは金輪際もう使ってはいけません。禁止します」

「え、私が前にいたプロジェクトでは `var` をけっこう見かけたんですが、ダメなんですか？」

「今の JavaScript には `const` と `let` という変数を宣言するためのキーワードが追加されてる。だからその2つを代わりに使って」

「わかりました。でもどうして `var` を使っちゃいけないんでしょうか？」

「従来からある `var` は、安全なプログラミングをする上で次のような3つの問題を抱えてるの」

1. 再宣言および再代入が可能
2. 変数の参照が巻き上げられる
3. スコープ単位が関数

「ふーむ。よくわかりませんが、これらの何がどうまずいんですか？」

「ひとつずつ見ていこうか。 `node` コマンドを REPL モードで使いながら説明するね」

```
$ node
> var a = 1;
> a = 2;          // 再代入OK
> a
2
> var a = 3;      // 再宣言OK
> a
3

> let b = 1;
> b = 2;          // 再代入OK
> b
2
> let b = 3;      // 再宣言はNG
Uncaught SyntaxError: Identifier 'b' has already been declared
```



```
> const c = 1;
> c = 2;           // 再代入はNG
Uncaught TypeError: Assignment to constant variable.

> const c = 3;    // 再宣言もNG
Uncaught SyntaxError: Identifier 'c' has already been declared
```

	再代入	再宣言
var	○	○
let	○	×
const	×	×

「var、let、const の再宣言と再代入の挙動のちがいがわかった？」

「はい、完璧です！」

「関数型プログラミングの文脈では、変数などを可変であることをミュータブル (mutable)、不変であることをイミュータブル (immutable) と表現する。let はミュータブルな、const はイミュータブルな変数を宣言するキーワードってことね」

「でも『イミュータブルな変数』というのは、なんか矛盾を含んだ表現ですよ」

「だから慣例的には『定数』と呼ぶほうが多いかな。なおコンパイル言語における定数とはコンパイル時に値が確定して、実行時にはその参照先も含めて変更不可のものを指す。いっぽうこちらは再代入が禁止されているというだけで、const で宣言した配列やオブジェクトはその要素が変更可能だから、定数と呼ぶのに違和感がある人も少なくないけど」

「ふーむ、なるほど」

「ちなみに var は『variable (変化しやすい)』、const は『constant (恒常的な)』が由来。『～させてやる』という意味の動詞 let が最初に変数の宣言に使われるようになったのは Lisp なので、その系列の Scheme を参考にして作られた JavaScript はその流れでこのキーワードが採用されたのかもね。ちなみに Rust や Swift など他の言語でも let を使うけど、その場合の変数はイミュータブルになる」

「……まぎらわしいですね」

「まあそのへんは JavaScript が古い歴史を引きずってる^{ゆえん}所以だよな。

話を戻すと、var < let < const と後のものを使ったほうがより安全なコードが書ける。特に再宣言は他の多くの言語では許されていない書き方なので、潜在的なバグを生みやすい。これが var を使ってはいけない理由のひとつめ」

「なるほど」

「var の問題の 2 つめ『変数の参照が巻き上げられる』については、まずその挙動から見てもらおうかな。次のコードを実行してみて」

リスト 9: 02-vars/hoisting.js

```
a = 100;
console.log(a);

var a;
$ node 02-javascript/02-vars/hoisting.js
100
```

「んん？ どこかおかしいですか？」

「ああ、Ruby や PHP は変数の代入が宣言を兼ねるので、それに慣れてると違和感が少ないのかな。よく見てみて。変数 `a` を宣言する前に代入できてしまってるよね。これが `let` や `const` なら参照エラーになるところだけど、`var` では許されるの。これを巻き上げ (Hoisting) ⁵⁹というんだけど、これも他の言語ではなかなか見られない仕様なので、開発者がミスリードされてうっかりバグを作り込みかねない。Stack Overflow でもよく初心者の質問のネタになってる」

「そうなんですね」

「var の問題の最後は『スコープ単位が関数』。私としてはこれがいちばんやっかいだと思ってる。とりあえずこのサンプルコードを見てみて。結果はどうなると思う？」

リスト 10: 02-vars/scope.js

```
var n = 0;

if (true) {
  var n = 50;
  var m = 100;
  console.log(n);
}

console.log(n);
console.log(m);
```

⁵⁹ 「Hoisting (巻き上げ、ホイスティング) - 用語集 | MDN」
<https://developer.mozilla.org/ja/docs/Glossary/Hoisting>

「最後から2行めの `console.log(n)` の出力は `0` ですよ。次の行は `m` を参照できないんじゃないでしょうか。だから `50`、`0` ときて最後は参照エラーになると思います」

「ふふふ、じゃあ実行して確かめてみようか」

```
$ node 02-javascript/02-vars/scope.js
50
50
100
```

「えっ！ `50`、`50`、`100`？」

「まあ奇妙に思うよね。なぜこんな挙動になるかという JavaScript では `var` で定義された変数のスコープって関数単位なんだよ。だから `if` のような制御構文のブロックをすり抜けてしまう。いっぽう Ruby を始めとする大多数の言語の変数スコープはブロック単位だよ。最初に秋谷さんが想定した挙動は、ブロックスコープを前提としたものだった」

「そうです」

「それじゃ、このコードの `var` をすべて `const` に書き換えて実行し直してみて。最初に秋谷さんが言ってくれた通りの挙動になったでしょう？」

「.....ほんとだ」

「他の言語に慣れた大半の開発者にはブロックスコープのほうが直感的だよ。だから `var` の関数スコープは評判が悪かった。ES2015 で `const` と `let` が導入されたときブロックスコープになったのは当然の流れだったといえる。

以上の3つが `var` を使うことを私が許さない理由。どれもこれもバグの原因になりかねないですよ。納得してもらえた？」

「はい、納得しました。こういうことでしたらもちろん、もう `var` は使いません！」

「うん、素直でよろしい。ただ `var` だけでなく、意図しない値の上書きもバグの原因になるので `let` も気軽に使わないようにね。あくまで `const` が第一選択肢で、どうしても再代入が必要なときだけ `let` にするってことを徹底してほしいの」

「了解です！」

2-3. JavaScript のデータ型

2-3-1. JavaScript におけるプリミティブ型

「次は JavaScript のデータ型について」

「ん？ JavaScript って Ruby とかと同じく、型がない言語じゃないんですっけ」

「その『型がある／ない』という言い方は誤解を招くので使ってほしくないんだよね。JavaScript も Ruby も**静的型付け言語**ではないけど**動的型付け言語**であって、ちゃんと型を持ってるよ」

「……うーん。その『静的型付け』と『動的型付け』って、何がどうちがうんですか？」

「まず両者ともデータ値そのものに型があることは共通してる。異なるのは静的型付け言語は、変数や関数の引数および戻り値の型がプログラムの実行前にあらかじめ決まっていなければならないのに対して、動的型付け言語ではそれらが実行時の値によって文字通り**動的に変化**するということ。静的型付け言語である TypeScript と挙動のちがいを比べてみるとわかりやすいかな」

```
$ node
> let num = 100;
> num = 'foo';
'foo'

$ ts-node
> let num: number = 100;
> num = 200;
200
> num = 'foo';
<repl>.ts:6:1 - error TS2322: Type 'string' is not assignable to type 'number'.

num = 'foo'
~~~
```

「前者が動的型付け言語である JavaScript の挙動。これは Ruby と同じなのでわかると思うけど、一度宣言した変数に異なるデータ型の値を入れ直すことができる。後者が静的型付け言語の TypeScript で、変数を `let` で宣言していてもデータ型が異なると値の再代入はコンパイルエラーになって許されない」

「なるほど、理解できました。軽々しく『型がない』とか言っちゃいけないんですね……」

「わかってもらえたようね。そして JavaScript では Java と同様、データ型が**プリミティブ型**と**オブジェクト型**の2つに大別される⁶⁰。値がプリミティブであるというのは、それがオブジェクトではな

⁶⁰ 「JavaScript のデータ型とデータ構造 - JavaScript | MDN」
https://developer.mozilla.org/ja/docs/Web/JavaScript/Data_structures

い、インスタンスメソッドを持たないデータだということ。JavaScript のプリミティブ型は今のところ次の7種類がある」

- **Boolean 型**（論理型）
…… `true` および `false` の2つの真偽値を扱うデータ型。
- **Number 型**（数値型）
…… 数値を扱うためのデータ型。他の多くの言語と異なり、整数も小数も同じ **Number 型** になる。扱うことができる最大値は $2^{53} - 1$ (9,007,199,254,740,991 $\approx$ 9 千兆)。
- **BigInt 型**（長整数型）
…… **Number 型** では扱いきれない大きな数値を扱うためのデータ型。ES2020 で追加された。**Number 型** と互換性がなく、相互に代入や計算、等値比較などは行えない。
- **String 型**（文字列型）
…… 文字列（テキストを表す連続した文字）を扱うためのデータ型。
- **Symbol 型**
…… 「シンボル値」という固有の識別子を表現する値。ES2015 から導入された。`Symbol()` 関数を呼び出すことで動的に生成されるが、基本的に同じシンボル値を後から作成できない。オブジェクトのプロパティキーとして使用可能。
- **Null 型**
…… プリミティブ値 `null` は何のデータも存在しない状態を明示的に表す。
- **Undefined 型**
…… プリミティブ値 `undefined` は宣言のみが行われた変数や、オブジェクト内の存在しないプロパティへのアクセスに割り当てられる。他の多くの言語と異なり、`null` と明確に区別される。

「これらの内 **BigInt 型** は扱える環境がまだ限定的だったり、**Symbol 型** は JSON でのパースができなかったり取り扱いが不便なのであまり活用場面がない。だから私たち開発者がよく使うのはこれらを除いた、あとの5種類ね」

「なるほど。これらの型で値の真偽がどうなってるか教えてくださいか？」

「いい質問だね。falsy なのがそれぞれ `false`、`0`、`NaN`、`''`（＝空文字）、`null`、`undefined` で、それ以外は全部 `truthy` になるよ」

「ちょっと待ってください。NaN って？」

「**Number 型** でありながら、数値ではないことを示す値のこと。『Not a Number』の略ね。発生条件としては次のものがある」

- `0` 同士の除算
- `NaN` を含んだ演算

- その他の無効な演算および処理

「最後の『その他の無効な演算および処理』とは具体的には？」

「たとえばこんなものが挙げられる」

```
> Math.sqrt(-1)    // -1の平方根
NaN
> Infinity * 0     // 無限大と0の乗算
NaN
> parseInt('foo')  // 数字以外の文字列を数値としてパース
NaN
```

「ふむふむ、たしかにこれらは数値で表現できませんね」

「ちなみにある値が NaN かどうかの判定は `Number.isNaN()` を使う。JavaScript には他にグローバルな `isNaN()` 関数があるんだけど、これは使っちゃだめ。数値以外の値を渡すと、暗黙の型変換が起って判定結果がおかしくなるので」

```
> Number.isNaN(Infinity * 0)
true
> Number.isNaN(1234)
false
> Number.isNaN('foo')
false
> isNaN('foo')
true
```

2-3-2. プリミティブ値のリテラルとラッパーオブジェクト

「ところでさっき柴崎さん、プリミティブ型の定義の中で『インスタンスメソッドを持たない』っていいましたよね。でも手元で確認してみたら、文字列とか普通にメソッドが使えちゃえますよ。本当は JavaScript も Ruby のようにすべてのデータがオブジェクトなんじゃないんですか？」

```
> 'Serval lives in savanna'.replace('savanna', 'jungle');
'Serval lives in jungle'
```

「うーん、その挙動に気づいちゃったか。これを理解するにはちょっと入り組んだ説明が必要になるのでよく聞いてね。まずプリミティブ型の値を定義するには、通常**リテラル** (Literal) ⁶¹を使う。literal は『文字通りの』という意味の言葉で、プログラミング言語においてはソースコードに数値や文字列をベタ書きしてその値を表現する式であることからこう呼ばれるのね。各プリミティブ型に用意されてるリテラルは次のとおり」

- **Boolean 型**

…… `true` と `false` の2つの**真偽値リテラル**がある。

- **Number 型**

…… `36` や `-9` のように数字を記述する**数値リテラル**。先頭に `0x` をつけることで **16進数**、`0o` をつけることで **8進数**、`0b` をつけることで **2進数** が表現される。他に `3.14` や `2.1e8` などの形式で表現する**浮動小数点リテラル**がある。また `1_000_000` のように途中に挿入された非連続のアンダースコアは無視される。

- **BigInt 型**

…… `100n` のように整数の後ろに `n` をつけて表現する**長整数値リテラル**。

- **String 型**

…… シングルクォート、またはダブルクォートで囲まれた**文字列リテラル**。バッククォートで囲むと、改行を含む複数行テキストや `${}` による式の展開が可能な**テンプレートリテラル**⁶²になる。

- **Null 型**

…… **Null リテラル** である `null` は、プリミティブ値 `null` を返す。

「ちなみに、ややこしいけど `undefined` はリテラルじゃなく、プリミティブ値 `undefined` が格納されている `undefined` という名前のグローバル変数ね」

「へー、ここまで特に意識せずを書いてましたけど、これらはそういう構文として規定されてたんですね」

「うん。それでね、ここに挙げたリテラルを用いて定義された値は、まちががなくプリミティブ型であってオブジェクトではないの。だからさっき秋谷さんが挙げてくれたサンプルの文字列もプリミティブ型なわけ」

「ならどうしてそれらの値から直接メソッドが生えるんですか？」

「まず `null` と `undefined` を除くすべてのプリミティブ型には、それらの値を包含する**ラッパーオブジ**

⁶¹ 「文法とデータ型 - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Guide/Grammar_and_types

⁶² 「テンプレートリテラル (テンプレート文字列) - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Template_literals

エクト (Wrapper Object) というものが存在してる。String 型なら `String`⁶³、Number 型なら `Number`⁶⁴ がそれに相当する。これらは JavaScript の言語処理系に標準組み込みオブジェクト⁶⁵として備わってる。

ただ同じ値を表現するものであっても、プリミティブ値とそのラッパーオブジェクトは等価とはならない。ラッパーオブジェクトに備わってる `valueOf()` というメソッドが返す値がそのプリミティブ値に対応するの。実際の挙動で示すとこうなるね」

```
> const str1 = 'Serval';
> const str2 = new String('Serval');
> str1 === str2
false
> str1 === str2.valueOf()
true
```

「そして JavaScript には、プリミティブ型の値に対してアクセスするとき、その対応するラッパーオブジェクトに自動変換するという仕様があるのね。だからさっき秋谷さんが挙げてくれた例は、内部でこのように変換されてる」

```
'Serval lives in savanna'.replace('savanna', 'jungle');
↓
(new String('Serval lives in savanna')).replace('savanna', 'jungle');
```

「`replace()` が実行できてるのは、変換された `String` オブジェクトのインスタンスメソッドを実行してたわけね」

「.....そういうことだったんですか。なるほど、納得しました」

⁶³ 「String - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/String

⁶⁴ 「Number - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Number

⁶⁵ 実行環境にあらかじめ組み込まれており明示的にインポートすることなく使うことができるオブジェクトのうち、ECMAScript の標準仕様として規定され用意されているオブジェクト。Ruby の組み込みライブラリ、Java のコアライブラリに相当する。なお JavaScript では `parseInt()` や `encodeURIComponent()` といったグローバル関数も標準組み込みオブジェクトに含まれる。

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects

第3章 関数型プログラミングでいこう

3-1. 関数型プログラミングは何がうれしい？

「はい、次はお待ちかねの関数型プログラミング講座です」

「ううっ、難しそう……」

「まあまあ、そうかまえないで。まずは基礎知識の確認から始めていこう。いま当たり前のように使った用語だけでも、そもそも秋谷さんは『**関数型プログラミング** (Functional Programming)』ってどういうものを意味するんだと思う？」

「……難しい質問ですね。私が知ってる Ruby も Java も関数型言語じゃないものの、関数型プログラミングっぽい機能は部分的に取り入れられてるっていいですよ。たとえば Ruby なら `Enumerable` モジュール¹¹⁶のメソッドの `map()` とか `reduce()` とか。ただ、どうしてこれらが関数型っぽいといわれるのかはよくわかってませんけど」

「じゃあそのへんから説明していこう。まず関数型プログラミングでいう『関数』は、Java や Python における一連の手続き処理をカプセル化したあれのことを指すのではなく、より数学的な意味合いが強いものなの。 $y = f(x)$ ってやつね。数学の関数は、与えた引数が同じなら返る値は常に一定でしょう？ そういう同じ入力に対して同じ作用と同じ出力が保証されていることを**参照透過性**という。関数型プログラミングとは、参照透過的な関数を組み合わせることで解決すべき問題に対処していく**宣言型**のプログラミングのスタイルのことをいうの」

「宣言型？ また知らない言葉がでましたけど」

「そうね、じゃあ各種プログラミング・パラダイムの整理をしておこうか。プログラミングのパラダイムはまず大きく『**命令型プログラミング** (Imperative Programming)』と『**宣言型プログラミング** (Declarative Programming)』の2つに大別される。命令型とは、最終的な出力を得るために状態を変化させる連続した命令文によって記述されるプログラミングスタイルのことをいう。ステップ順にその指示にしたがっていけば、お目当ての料理ができあがるレシピと似てるね。目の前のことだけ考えてひとつずつ積み上げていけばいいので、プログラミング初心者にとっても敷居が低い」

「はい、直感的だと思います」

「その命令型に分類される代表的なパラダイムが『**手続き型プログラミング** (Procedural Programming)』。手続きというの是一連の命令文の組み合わせをまとめた、サブルーチンや関数と呼ばれるもの。主にグローバル変数としてまとめたデータを、複数の手続きを経て最終結果を得るプログラミングスタイルね。」

¹¹⁶ <https://docs.ruby-lang.org/ja/latest/class/Enumerable.html>

命令型と手続き型はほぼ同じ意味で用いられがちなんだけど、命令型のほうが大きい概念で、現代で『手続き型言語』と明記されるのは命令型かつオブジェクト指向から距離を置いているものを指すことが多いね。COBOL や BASIC、Pascal、C といった比較的古い言語が該当するけど、Go もこれに分類される」

「オブジェクト指向は手続き型に含まれないんですか？」

「オブジェクト指向プログラミングは、命令型か宣言型かという分類からは独立したパラダイムなの。構造化されたデータとその振る舞いをカプセル化したもの同士を相互に作用させて最終結果を得るスタイルだけど、手続き型のスタイルとは矛盾しない。このオブジェクト指向と手続き型、両者の性質を兼ね備えた言語が現代ではもっともメジャーになってるといえる。Java、C++、C#、Python、Ruby とかがそうだね」

「へー、そういう分類になってたんですか」

「で、最初に戻ってその一大勢力となってる命令型と対立する概念が宣言型ね。宣言型プログラミングでは出力を得る方法ではなく、出力の性質・あるべき状態を文字通り**宣言**することでプログラムを構成する」

「そんなことができるんですか？」

「具体的には、SQL がもっとも有名な宣言型プログラミング言語にあたるかな。SQL のクエリは『どのようにデータベースにアクセスしてデータを取得してくるか』という手続きではなく、『どんなデータが欲しいか』を宣言することで出力が得られるでしょ」

リスト46: SQL の記述例

```
SELECT * FROM employees WHERE employee_number < 1000 AND gender = 'f' ORDER BY hire_date;
```

「ああ、たしかに！でも SQL で長いプログラムを書くことはできませんし、データベースへの問い合わせという限定的な役割だからこそそれが可能なんじゃないでしょうか」

「そのどうやって宣言型で複雑な処理を行わせるかという問題に対するひとつの解こそが、これから学ぼうとして関数型プログラミングなのよ。秋谷さんのように手続き型言語しかさわったことがない人にとって難解に見えるのは、出力のあるべき状態を数学的に定義する手法になじみがないだけだと思う」

「むむむむむ.....。でも Lisp のサンプルコードとか見たことがありますけど、かっこだらけで意味不明だったんですよ」

「うん、Lisp は関数型言語としては最古の部類に入る代表的なものだね。なぜ Lisp のコードがかっこだらけなのかというのにはちゃんと理由がある。そこも含めて、具体的なコードで説明していこう。同一の処理を手続き型と関数型でそれぞれ実装したコードを比較してみる。1 から 100 までの範囲における 8 の倍数を格納した配列を作るという簡単な処理を、それぞれ 2 つのアプローチで書き分けると次のようになる」

リスト47: 01-advantages/procedural-vs-functional.js

```
// 手続き型プログラミング
{
  const octuples = [];

  for (let n = 1; n <= 100; n += 1) {
    if (n % 8 === 0) {
      octuples.push(n);
    }
  }

  console.log(octuples);
}

// 関数型プログラミング
{
  const range = (start, end) => [...Array(end - start)].map( (_, n) => n + start);
  const octuples = range(1, 101).filter((n) => n % 8 === 0);
  console.log(octuples);
}
```

「ああー、これこれ。この後者みたいのが関数型っぽいんですね。でも最初の `range` の定義では何やってるかさっぱり見当つきませんけど……」

「ああこれはね、JavaScript には Ruby の `1..100` のような範囲演算子や Python の `range()` 関数に相当するものがないので、即席で作ってるんだよ。Lodash の `range()`¹¹⁷ をインポートしてきたりすればそこは省略できるので、実質メインの処理はその下の行だけとっていいかな。Ruby だったら `p (1..100).filter { |n| n % 8 == 0 }` 一発だね」

「ふむふむ、Ruby やっぱすごい！ それに 1 行で `range()` 関数を作ってしまう柴崎さんもすごい！」

「ふふふ、いやそれほどでも……。それでね、こうして 2 つのスタイルを対比させるとそれぞれの異なる特徴が浮かび上がってくる。

まずひとつめが**可変性** (Mutability) と**不変性** (Immutability)。関数型で書いたほうは変数のひとつたりとも再代入や破壊的変更がなされてないのに対して、手続き型のほうは `octuples` に 12 回の破壊的変更、`n` に 100 回の再代入がある」

「あれっ、`n` もその内に入っちゃうんですか？」

「`for` 文による定型処理とはいえ繰り返しのたびに再代入されてるし、ループブロックの中でも任意に上書き可能だからね。そして可変性の何が問題かという、ミュータブルな変数に依存する処理が

¹¹⁷ <https://lodash.com/docs/#range>

多ければ多いほど、プログラムの予測可能性が低下してバグが入り込みやすくなる。最終的な値が処理ステップの順序に依存しているため、ちょっとした見落としで踏むべきだったステップが飛ばされたり、意図せぬ別のステップが紛れ込んだりするようなことが起こる」

「……うーん、そういわれればそうかもしれませんけど」

「^{はな}端からイミュータブルな変数しか使ってなければそういうことは起きないよね。ここがまず関数型プログラミングの優れているポイントといえるね」

「ふーむ、安全性か……」

「イミュータビリティ、つまり不変性を守ることでプログラムから副作用が排除される。副作用とはあるリソースの変更がアウトプットに影響を与えてしまう、関数においてはその参照透過性を壊してしまうこと。ミュータブルなリソースがプログラムの中に存在していれば、それだけ副作用を生みやすいわけ。

さらに関数型プログラミングでは、代入自体もできるだけ控える傾向にある。最初から代入しなければ壊れるものもないからね。メソッドチェーンや演算子で式をつないでいき、関数もその場限りのものならできるだけ無名関数のまま使う」

「なるほど」

「対照的な特徴の2つめとして、手続き型が**文** (Statement) を多用するいっぽう、関数型では**式** (Expression) を組み合わせてプログラムを構成する¹¹⁸。サンプルでの手続き型のほうは `for` 文によるループの中で `if` 文によって条件分岐をしてというように、値を返さないサブルーチンの中で計算が実行されてるよね」

「はい、私もよくやってる書き方ですけど」

「関数型のほうはすべてが値を返す式の組み合わせで、それが左辺から右辺へ評価されていき最終的な値へ到達する形になってる。そのおかげでコードが非常にシンプルになる。経験が浅いと数式の羅列にひるんでしまうかもしれないけど、`range` 式が返す値でひと区切り、`filter` 式が返す値でひと区切りと、それぞれの式は独立していて相互に干渉することがないので、一度に考えることも少なくて済むんだよ」

「へー、そういうもんですか」

「対照的な特徴の最後3つめとして、これはまあ印象的なものだけど、手続き型ではボトムアップ的に積み上げていって最終成果物を完成させる。最初 `octuples` は空配列だったのが、要素がひとつずつ追加されていって完成に至ってるよね。

いっぽう関数型では、最初から完成形を見据えた上で大雑把なところから絞り込んでいく形になる。このサンプルではまず1から100の整数配列を作った上で、そこから8で割り切れるものだけを抜き出してるよね」

¹¹⁸ 文と式のちがいについては「2-4-1. 関数宣言と関数式」を参照のこと。

「ああ、たしかに！ アプローチが逆ですね！」

「そう。手続き型プログラミングが終わりが見えず似たような面がひたすら繰り返されるスペースインベーダーだとすれば、関数型プログラミングは開始早々マップの対岸にラスボスである竜王の城がすでに見えてるドラクエ1みたいなイメージだね」

「ええ……、その^{たと}えもわかりやすいんだか、わかりにくいんだか……」

「……コホン。話を戻すと、2つを比べてみても完成形を見据えて絞り込まれていく関数型のコードのほうが、読むほうもだけど書くほうも迷子になりづらいよね。まあ書く人のセンスによるところもあるんだけど、熟練のプログラマが書く関数型のコードは意図がはっきりしてて可読性が高いの。条件分岐やループであっちに行ったりこっちに来たりしないからね」

「ふーむ、熟練の職人による技はシンプルだからこそ美しいわけですね」

「以上が、関数型プログラミングと手続き型プログラミングのちがいと、関数型プログラミングのどこが優れているかっていう私なりの説明。理解できた？」

「はい、かなり頭が整理されました！　ところで質問なんですけど、この関数型プログラミングで書かれたサンプルコードはJavaScriptで書かれてるじゃないですか。ということはJavaScriptは関数型言語っていう認識でいいんでしょうか？」

「いい質問だね。関数型言語というものを厳密に定義するのは難しいけど、少なくとも次の2つの条件を満たしてる必要があるだろうね」

- 第一級関数をサポート
- 文ではなく式が主役

「第一級関数って前にも出てきましたよね。えーっと……」

「関数をデータ構造に含めることができ、さらに他の関数に引数として渡したり、戻り値として返したりできる性質のことだね。JavaやRubyは関数型プログラミングっぽいことができてこの条件を満たさないからまずふるいにかける」

「なるほど。次の『式が主役』というのは？」

「文で書けなくもないけど、言語の思想として式を主体に書くようになってることね。Lispはかっこだらけと秋谷さんは言ってたけど、それはLispのプログラミングは複数の式の組み合わせで成り立っていて、ひとつひとつの式の区切りとしてかっこを使うのが合理的だと考えているからだね」

「へー、Lispのかっこだらけの構文にはそういう意図があったんですか」

「ScalaやHaskellではifも値を返す式だって前にも言ったけど、文法的にC系列につながる言語であっても、関数型言語を標榜しているものはそうやって式を主体に書けるようになってる。PythonやJavaScriptは第一級関数を備えながらも関数型言語だと言われないのはそれが理由」

「ふむふむ」

「現在よく使われている言語はマルチパラダイムといって複数のパラダイムの性質を併せ持っている

ものが多いから、なかなか明確に分類するのは難しいんだけどね。それでもおおまかにはこんな感じになるかな」

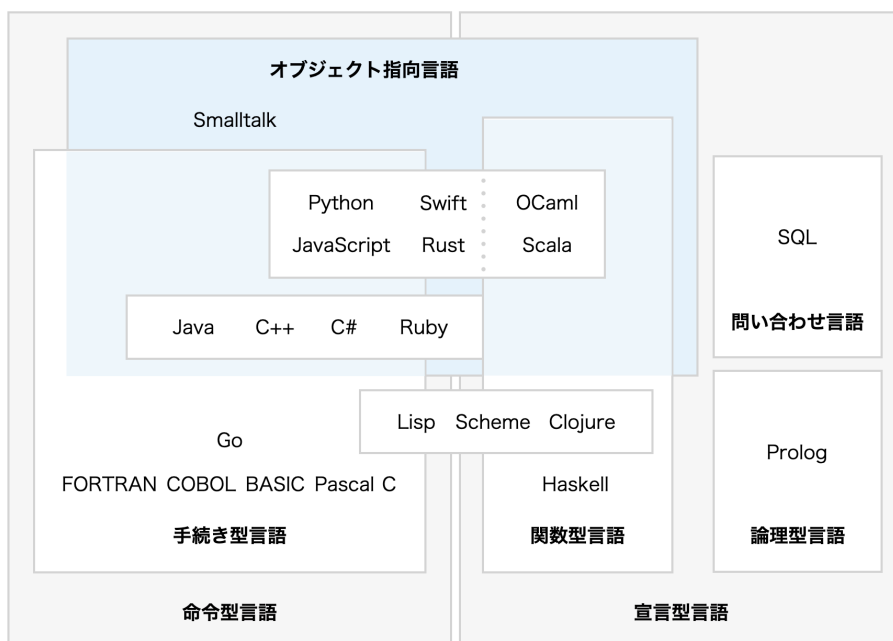


図 5: プログラミング言語の分類図

「JavaScript は明確に関数型言語には分類できないけど、第一級関数をサポートしてるから関数型プログラミングをやろうと思えばできるんですね。でも手続き型でも書けちゃうんなら、私はわかりやすいからそっちで書くことが多くなっちゃいそうです」

「関数型で書くことのメリットは実践を積み重ねていかないと難しいと思うけど、説得力を持った説明が欲しいならこんな本があるよ。現役のエンジニアによるおすすめ本リストによく入ってる『プログラマが知るべき 97 のこと』¹¹⁹という有名な本でね。内容は世界で活躍する 97 人の著名なプログラマが自分の大事にしている価値観をそれぞれの想いで語ってるエッセイ集なんだけど、その冒頭 2 番めに『関数型プログラミングを学ぶことの重要性¹²⁰』っていう文章がある」

「へえ——」

¹¹⁹ Kevlin Henney (編)・和田卓人 (監修)・夏目大 (訳) (2010) 『プログラマが知るべき 97 のこと』オライリー・ジャパン p.4

¹²⁰ 「関数型プログラミングを学ぶことの重要性 | プログラムが知るべき 97 のこと」
<https://プログラマが知るべき97のこと.com/エッセイ/関数型プログラミングを学ぶことの重要性/>

「そこに参照透過性がいかに大事か、不具合を減らしデバッグを簡単にするためにどういうコードを書くべきかというのが^{とうとう}語られてる。私もかなり共感するエッセイで、2 ページ程度の分量ですぐ読めるから見ておくといいよ。ネットにも全文掲載されてるけど、あとで本貸してあげるから」

「わかりました、読んでおきます！」

3-2. コレクションの反復処理

3-2-1. 配列の反復処理

「最初のサンプルでも使ってたけど、関数型プログラミングへの最初の入り口はコレクションの反復処理かな。まず Array オブジェクトのプロトタイプメソッドの中から、反復処理を行う主なものをピックアップして紹介していくね。これも実際に使ってるコードとその実行結果を見てもらったほうがわかりやすいと思う」

リスト48: 02-collection/array-iterate.js

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9];

console.log(
  arr.map((n) => n * 2),           // [ 2, 4, 6, 8, 10, 12, 14, 16, 18 ]
  arr.filter((n) => n % 3 === 0), // [ 3, 6, 9 ]
  arr.find((n) => n > 4),          // 5
  arr.findIndex((n) => n > 4),     // 4
  arr.every((n) => n !== 0),       // true
  arr.some((n) => n >= 10),        // false
);
```

- `map()`¹²¹ 対象の配列の要素ひとつひとつを任意に加工した新しい配列を返す
- `filter()`¹²² 与えた条件に適合する要素だけを抽出した新しい配列を返す

¹²¹ 「Array.prototype.map() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/map

¹²² 「Array.prototype.filter() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/filter

- `find()`¹²³ 与えた条件に適合した最初の要素を返す。見つからなかった場合は `undefined` を返す
- `findIndex()`¹²⁴ 与えた条件に適合した最初の要素のインデックスを返す。見つからなかった場合は `-1` を返す
- `every()`¹²⁵ 「与えた条件をすべての要素が満たすか」を真偽値で返す
- `some()`¹²⁶ 「与えた条件を満たす要素がひとつでもあるか」を真偽値で返す

「一気に説明するとこんな感じかな。だいじょうぶそう？」

「ところどころ Ruby と同じなので、だいたいわかります」

「よし、じゃ次いつてみよう。ちょっと難易度が高くなるよ。メソッドの引数として渡す関数に2つの引数が存在する `reduce()`¹²⁷ と `sort()`¹²⁸ ね」

リスト49: 02-collection/array-iterate2.js

```
const arr = [1, 2, 3, 4, 5];

console.log(
  arr.reduce((n, m) => n + m),          // 15
  arr.sort((n, m) => n > m ? -1 : 1),   // [ 5, 4, 3, 2, 1 ]
);
```

「`reduce()` って Ruby にもありますけど、いまいちよくわかってないんですよ.....」

「引数に何が入ってくるかというのをおぼえておくといいよ。この第2引数 `m` には `arr` の各要素の値が順番に入ってくるんだけど、第1引数 `n` には前回の関数の実行結果が入ってくる。そうして最終的

¹²³ 「Array.prototype.find() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/find

¹²⁴ 「Array.prototype.findIndex() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/findIndex

¹²⁵ 「Array.prototype.every() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/every

¹²⁶ 「Array.prototype.some() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/some

¹²⁷ 「Array.prototype.reduce() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/reduce

¹²⁸ 「Array.prototype.sort() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/sort

に `reduce()` が返すのは最後に実行された値だけってこと」

「……えーっと、待ってください。1 回ずつ実行結果を書き出してみますね」

- 1 回目の実行： `m = 1`、前回の実行がないので、結果は `1` がそのまま返る
- 2 回目の実行： `m = 2`、前回の実行結果により `n = 1`、結果は `1 + 2 = 3` が返る
- 3 回目の実行： `m = 3`、前回の実行結果により `n = 3`、結果は `3 + 3 = 6` が返る
- 4 回目の実行： `m = 4`、前回の実行結果により `n = 6`、結果は `6 + 4 = 10` が返る
- 5 回目の実行： `m = 5`、前回の実行結果により `n = 10`、結果は `10 + 5 = 15` が返る

「はい、よくできました」

「わあ、うれしい！ `reduce` ってただ『全要素を足すときに使う』ぐらいの意識だったので、ちゃんと理解できてスッキリしました。言葉の意味もよくわからないし」

「`reduce` の語源は『`re`（ふたたび、元に、後退）』 + 『`duce`（導く）』。辞書を引くと単に『減らす』としか載ってないこともあるけど、もっと具体的には生殖細胞の減数分裂とか、スープを煮詰めていくイメージだね」

「へえー、これでもう忘れることはなさそうです！」

「じゃ次、`sort()`。これは読んでそのとおりのソート（並べ替え）を行うメソッドで引数は省略することもできる。ただその場合は、各要素が暗黙のうちに文字列に変換され Unicode の文字コード順にソートされるというよくわからない仕様なので、極力省略しないほうがいい。

省略しない場合の引数には、比較関数というものを渡すようになってる。その比較関数が守るべきルールは次の 3 つ」

- i. 第 1 引数が第 2 引数より優先度が高い（前に並べる）場合、0 未満の数値を返す
- ii. 第 1 引数が第 2 引数より優先度が低い（後に並べる）場合、0 より大きい数値を返す
- iii. 第 1 引数と第 2 引数の優先度が同じ（ソートの必要がない）場合、0 を返す（※省略可）

「上の例で渡されてる比較関数は `(n, m) => n > m ? -1 : 1` だから、`n` が `m` より大きい場合に -1 を返すルール i が与えられ、それ以外の場合は 1 を返すルール ii が与えられてるわけですね。よって大きい順に並べ替えられてる」

「そうだね。比較関数は上記のルールを遵守さえしていれば、もっと複雑な条件を適用することももちろんできる。たとえばユーザーを生年月日順に並べるとか、投稿をコメントの多い順に並べるみたいなね」

「そっか、そういう使い方もできるんですね」

「ところで JavaScript では `Array` オブジェクトの `sort()` は破壊的メソッドなので、他の処理と組み合わせる際には気をつける必要がある」

リスト 50: 02-collection/destructive.js

```
const arr = [4, 8, 2, 6];

console.log(
  arr.sort((n, m) => n < m ? -1 : 1),    // [2, 4, 6, 8]
  arr,                                  // [2, 4, 6, 8]
);
```

「えっ、これやばくないですか？ それにこれ見て思いましたが、JavaScript って破壊的メソッドと非破壊的メソッドが混在して見分ける方法がないのひどいですね。Ruby なら破壊的メソッドには `sort!` みたいに末尾に `!` がつくのでわかりやすいのに」

「うん、例によって歴史的経緯によってこんなまぎらわしくなっちゃってる。ただ配列に要素を追加し戻り値として新しい配列の長さを返す `push()`¹²⁹ や `unshift()`¹³⁰、配列から要素を取り除いてその値を返す `pop()`¹³¹ や `shift()`¹³² なんかは破壊的メソッドだけど、返す値が更新した配列じゃないので関数型プログラミングの文脈ではまず出番がないはず。だから `sort()` の他に気をつける必要があるのは、配列の要素を反転させる `reverse()`¹³³ くらい」

「なるほど。 `sort()` と `reverse()` はおぼえておくべき破壊的メソッド、と」

「うん。ただ ES2023 で処理が同じで非破壊的メソッドの `toSorted()`¹³⁴ と `toReversed()`¹³⁵ が追加されたの。だから ES2023 以降が有効な環境ではそっちを使うようにするといいよ」

「そうじゃない環境ではどうしたらいいのでしょうか？」

¹²⁹ 「Array.prototype.push() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/push

¹³⁰ 「Array.prototype.unshift() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/unshift

¹³¹ 「Array.prototype.pop() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/pop

¹³² 「Array.prototype.shift() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/shift

¹³³ 「Array.prototype.reverse() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/reverse

¹³⁴ 「Array.prototype.toSorted() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/toSorted

¹³⁵ 「Array.prototype.toReversed() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/toReversed

「JavaScript で配列の破壊的メソッドを、元の値を破壊せずに使える裏技がある。サンプルコードで紹介しよう」

リスト51: 02-collection/non-destructive.js

```
const list = [4, 8, 2, 6];

const sortedList = list.slice().sort((n, m) => n < m ? -1 : 1);
console.log(sortedList, list);    // [2, 4, 6, 8] [4, 8, 2, 6]

const reverseList = [...list].reverse();
console.log(reverseList, list);   // [6, 2, 8, 4] [4, 8, 2, 6]
```

「`slice(n, m)`¹³⁶ は配列の 0 で始まるインデックス `n` から `m - 1` までの選択された範囲をシャローコピーして新しい配列を返すメソッド。引数を完全に省略すると元の配列と全く同じものができるので、これを挟めば元の配列が破壊されることはなくなる。

もうひとつの方法は空配列の中に元の配列をスプレッド構文で展開するもの。どっちを使ってもいいよ」

「なるほど。私は後者のスプレッド構文を使うほうが好みですね。コード量も少ないし、何やってるかわかりやすいし」

「そうね。スプレッド構文なら配列の前後に要素を追加するときも `[0, ...list, 99]` のように書ける。基本はスプレッド構文で、配列の一部を切り出したいときは `slice()` と使い分けるのがいいかもね。

ここまでは必ず値を返す式だったけど、配列の反復処理には値を返さない構文もある。よく使われるのは `Array.prototype.forEach()` メソッド¹³⁷と `for...of` 文¹³⁸あたりかな」

リスト52: 02-collection/array-iterate-for.js

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9];

arr.forEach((n) => {
  if (n % 2 === 0) {
    console.log(`${n} is even`);
  }
})
```

¹³⁶ 「Array.prototype.slice() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/slice

¹³⁷ 「Array.prototype.forEach() - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/forEach

¹³⁸ 「for...of - JavaScript | MDN」

<https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Statements/for...of>

```
});  
  
for (let n of arr) {  
  if (n % 2 !== 0) {  
    console.log(`${n} is odd`);  
  }  
}
```

「これ、どっちを使ったほうがいいんですか？」

「まあ、あえていうなら `forEach()` かな。 `for...of` は Airbnb の JavaScript スタイルガイドでも使うなっていわれてるし¹³⁹。ただ2つを比べれば内部にミュータブルな変数が生じないのと、ここまで挙げた反復処理のメソッドとスタイルが共通してメソッドチェーンでつながられるから `forEach` がまだマシって程度のニュアンスでね」

「本来ならどっちも使わないほうがいいと？」

「うん。そもそも値を返さないこれらを使う目的って、主にその中で外部のミュータブルな変数を書き換えるといった副作用を起こすことだろうから、関数型プログラミングの文脈では往々にしてロジックの組み方をまちがえてるケースが多い。ほとんどの処理は `map()` や `find()` といった最初に挙げた値を返すメソッドでの組み合わせで十分なはずなので、どうしても他の方法が見つからなかった場合のみ `forEach()` を例外的に使うという感じだね」

「なるほど」

「あと `Array` で紹介しておきたいのが `includes()`¹⁴⁰ というメソッド。これは特に関数型的な処理を行うものではないんだけど、アプリケーション開発では活用できる場面がけっこうある」

リスト53: 02-collection/includes.js

```
const arr = [1, 2, 3, 4, 5];  
  
console.log(arr.includes(5)); // true  
console.log(arr.includes(8)); // false
```

「`includes()` は『指定した値の要素がひとつでも含まれているか』を真偽値で返す。条件式に活用できるので便利だよ。最後にもうひとつ `Array` を使った裏技を覚えておこう。JavaScript で任意の回数だけ繰り返し処理をしたい場合、外部のライブラリを使わずに手っ取り早く書く方法ね」

¹³⁹ 「Airbnb JavaScript Style Guide」 > 「11. Iterators and Generators」

<https://github.com/airbnb/javascript/#iterators-and-generators>

¹⁴⁰ 「`Array.prototype.includes()` - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Global_Objects/Array/includes

```

> [...Array(3)]
[ undefined, undefined, undefined ]

> [...Array(3)].map((_, n) => { console.log(`${n + 1} times`); });
1 times
2 times
3 times

> [...Array(3).keys()]
[ 0, 1, 2 ]

> [...Array(3).keys()].map((n) => { console.log(`${n + 1} times`); });
1 times
2 times
3 times

```

「実は `Array` オブジェクトは `keys()` メソッドを備えていて、イテレータ¹⁴¹という反復処理のための特殊なオブジェクトを返す。これを展開して配列に戻すと要素のインデックスが取り出せるので、こういう使い方ができるのね。言語設計者の意図とはちがう使い方だろうけど」

「Ruby でいう `Integer#times` メソッドみたいのが JavaScript にはないから、こういうアクロパティックな工夫が必要になっちゃうんですね」

「そう、`Range` オブジェクトもなければ `times` メソッドもない。そういう便利な記法が標準でそろってるところは Ruby がうらやましいね」

3-2-2. オブジェクトの反復処理

「次は狭義のオブジェクト¹⁴²、つまり標準組み込みオブジェクト `Object` を直接継承するオブジェクトの反復処理ね。巷では `for...in` 文¹⁴³が使われたコードをよく見かけるけど、配列のときの

¹⁴¹ 「イテレータとジェネレータ - JavaScript | MDN」

https://developer.mozilla.org/ja/docs/Web/JavaScript/Guide/Iterators_and_Generators

¹⁴² 「狭義のオブジェクト」と「広義のオブジェクト」のちがいについては「2-3-3. オブジェクト型とそのリテラル」を参照のこと。

¹⁴³ 「for...in - JavaScript | MDN」

<https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Statements/for...in>

第4章 TypeScript で型をご安全に

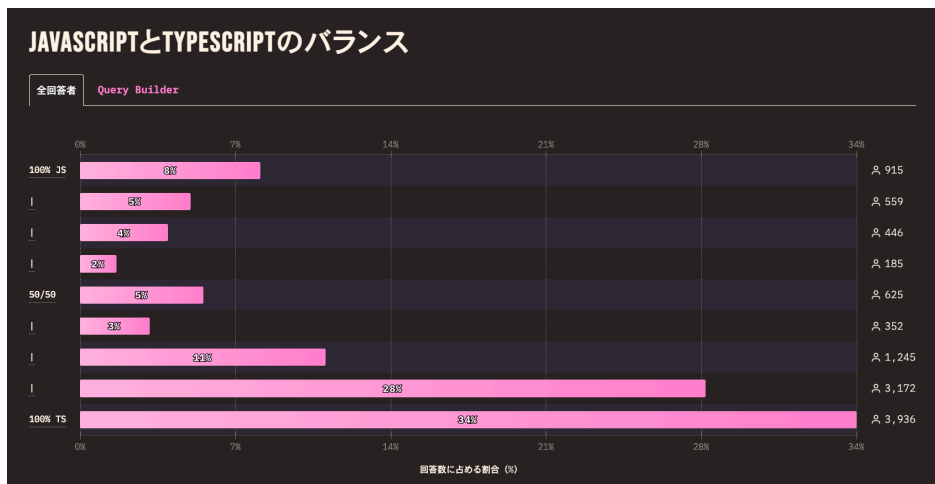
4-1. 時代が望んだ言語 TypeScript

「モダン JavaScript とそれによる関数型プログラミングの基本を押さえたところで、いよいよ TypeScript の説明に入ろうか」

「このチームのメイン開発言語なんですよ。でも疑問なんですけど、React は JavaScript で作られていて公式ドキュメントの解説中のコードもほとんど JavaScript なのに、どうして TypeScript を使うんですか？ 最新の ECMAScript ならモダンな仕様が盛り込まれているので、わざわざ TypeScript を使う必要性はないんじゃないでしょうか？」

「それについての理由はいくつかあるね。ただそれを説明する前に、現在の JavaScript コミュニティでどれだけ TypeScript が使われているかを認識しておいてもらおうかな。2018 年 10 月に npm の運営会社が発表した npm ユーザー約 16,000 人を対象にした調査¹⁶⁵では、その時点ですでに半数近くの 46 % が TypeScript を使っていると答えてる。さらに世界中の JavaScript ユーザーを対象に毎年行われている調査 State of JavaScript の 2024 年版では、回答者のうち 100% JavaScript で開発する時間の割合が 8% だったのに対し、TypeScript のそれは 34% と 4 倍以上になっている¹⁶⁶。

つまりコミュニティではもうかなり前から、TypeScript メインで開発する人のほうが JavaScript だけで開発してる人よりもずっと多いのよ」



¹⁶⁵ 「npm and the future of JavaScript」 <https://slides.com/seldo/npm-future-of-javascript>

¹⁶⁶ https://2024.stateofjs.com/ja-JP/usage/#js_ts_balance

図 6: JavaScript と TypeScript の使用率比較 (The State of JavaScript 2024 より)

「ええ——っ?!」

「そして Angular や Vue.js のようにフレームワーク自体が TypeScript で開発されていて、その使用を公式にサポートしているケースも多い。むしろ今となっては JavaScript しか公式にサポートしてない React のほうが少数派だね。Vite のようなツールチェーンも当然のように各フレーム向けに TypeScript を使ったテンプレートを用意してくれてるので、TypeScript でプロジェクトを始めるハードルは今や JavaScript とそう変わらない」

「……そうだったんですね。でもどうしてですか？ なぜ JavaScript の拡張言語のひとつでしかない TypeScript がここまで特別扱いされてるんでしょう？」

「古くから JavaScript ユーザーが言語に対して抱いていた最大の不満は、静的型付けがないことだったのね。それはアンケート結果にも顕著に現れている。そしてそのニーズにもっとも違和感の少ない形で答えてくれたのが TypeScript だったわけ」

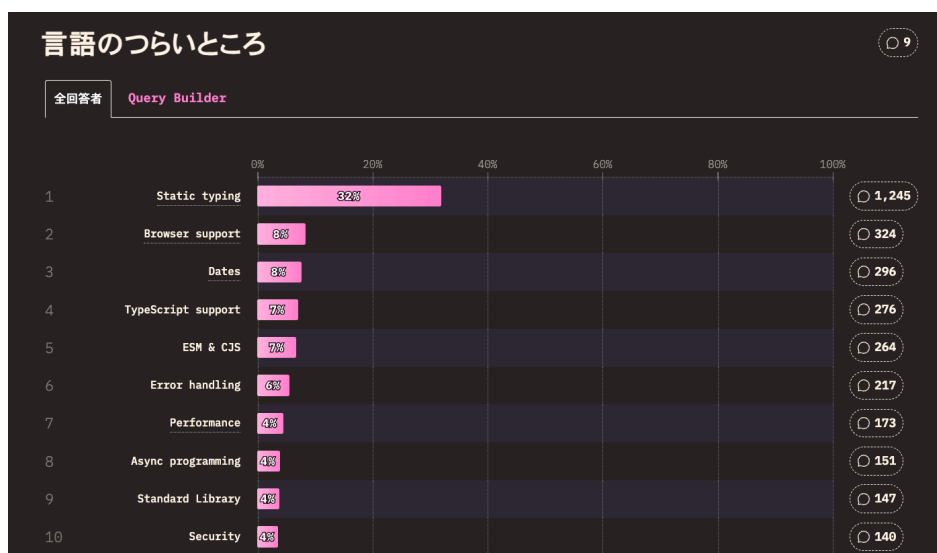


図 7: JavaScript のつらいところランキング (The State of JavaScript 2024 より)

「うーん、型ってそんなに必要ですか？ 今ひとつピンとこないんですけど……」

「Ruby 界隈の人は型を明示的に書かない動的型付けが好きだよねえ。でも考えてみて？ 2010 年代以降に登場し人気を集めている Go や Rust、Kotlin や Swift はすべて静的型付け言語だよ。時代のトレンドは確実に、動的型付け言語から静的型付け言語に移ってきてる」

「……言われてみればそうなんだろうけど」

「ひと昔前は Ruby や Python、PHP といった LL (Lightweight Language、軽量プログラミング言語) と呼ばれる動的型付け言語が人気だった。いちいち型を書かなくていい手軽さがウケて、そのほうが開発効率がいいと考えられたから。でも Web アプリケーションやモバイルアプリケーションに求められる機能が増え、開発が大規模化するにつれ、型に縛られていないことが逆に開発効率を落とすケースが増えてきた」

「そうなんですか？」

「Ruby で開発していると `NoMethodError` のような実行時エラーが起きることがめずらしくないよね。これはメソッドの引数や戻り値の型の整合が担保されてないせいで、想定しない値が渡されることがあるため。それを予防するためには、逐一テストを書いて型の検証をする必要がある。でも静的型付け言語であれば、その手間は省ける」

「わかりますけど、いちいち宣言時に型を書くのがめんどくさそうなんですよ」

「だから最近の静的型付け言語は、**型推論**という機能を備えてる。プログラマが型を書かなくても、言語処理系が文脈から推定して変数や関数に型をつけてくれるの。もちろんすべてのケースで省略できるわけじゃないけど、自分で型を書く労力をかなり減らしてくれる」

「へーえ」

「さらに静的型付け言語が開発者に人気の理由として、IDE¹⁶⁷ と相性がいいことが挙げられる。コーディング中にリアルタイムで型の不整合や null アクセスになる可能性のある部分を指摘してくれるし、前後の文脈から型が絞り込まれることで、入力補完の精度も格段に高まるの。それを知っているから JavaScript のユーザーは静的型付けの機能を熱望するわけ」

「静的型付けの重要性はわかりました。でもなぜ TypeScript だけが圧倒的に受け入れられているのかという疑問が残ります。JavaScript の拡張言語って他にも色々ありましたよね？」

「それについては長い歴史を踏まえないと納得できないだろうから、それを説明させて。ブラウザによるアプリケーション開発を実現する技術として JavaScript が開発されて 1997 年に ECMAScript として仕様が標準化された話はしたよね。その後 ES2 が 1998 年、ES3 が 1999 年に公開された後、2009 年まで次のバージョンが出なかった。この失われた 10 年の間に何もしていなかったというかとそうではなくて、大規模開発を見据えた野心的な仕様である ES4 を作成しようとしてたんだけど、各社の調整に失敗して破棄されてしまったの。だから次のバージョンは ES5 となっていて、ES4 は欠番になっている」

「幻の ES4 ですか.....。どんな仕様だったんですか？」

「モジュールや Java をそっくり模したようなクラス、それに静的型付けが含まれてた」

「えっ、JavaScript 本体に静的型付けが導入されてたかもしなかったんですか？ もし ES4 が採択

¹⁶⁷ 「Integrated Development Environment (統合開発環境)」の略。コンパイラ、テキストエディタ、デバッガなどをひとつの UI から利用できるようにしたツール。ここのニュアンスでは Visual Studio Code のような高機能エディタもそれに含まれます。

されていれば TypeScript は存在してなかったってこと？」

「歴史に if はないけど、ひょっとしたらそんな世界線もあるかもね。ただ ES4 は大きな破壊的変更を含んでいてそれまでの路線とはかけ離れていたため、破棄される運命だったのかもしれない。ES4 をもっとも推進していた Netscape 社が 2003 年に消滅、また当時 IE でブラウザの圧倒的シェアを握っていた Microsoft が複雑な実装を嫌って反対したのも大きかった¹⁶⁸。しかし ES4 を潰した張本人の

Microsoft が^{のち}後に TypeScript を生み出したのは歴史の皮肉としか言いようがないね」

「あ、そうか。TypeScript って Microsoft が開発してるんですね。それはどういう経緯だったんですか？」

「そのころ Microsoft は Office のような大規模なアプリをブラウザで動かしたいというニーズを持っていたのね。どれだけ大規模かっていうと、コードベースで数百万行レベルと当時の Web アプリの常識からはかけ離れてた¹⁶⁹」

「数百万行……。ちょっと想像つきませんね」

「いま普通に Office 365 がブラウザで動いてるけど、そこに至るまでは長い長い道のりがあった。2008 年に華々しくデビューした Google Chrome とかと比べると当時の IE の JavaScript エンジン は貧弱だったので、Microsoft はまずその改良を試み、新たに Chakra を完成させたの。そして Chakra チームが次に手掛けたのは、JavaScript による大規模アプリケーションの開発に耐えうる IDE の開発だった。Visual Studio Code のご先祖様にあたる。しかしコードベースが 7,000 行を超えたあたりで行き詰まってしまった。動的型付け言語である JavaScript では型情報によるコーディング支援が受けられなかったため、変数名をまちがえたりしてバグを見つけるのに半日もかかる有様だったとか」

「それはストレスが溜まりそうですね。……なるほど、だんだん話が見えてきました」

「そのまま開発を進めることに限界を感じた彼らは、JavaScript の言語そのものをどうにかする必要に迫られた。そこで相談したのが同社のテクニカルフェローだった Anders Hejlsberg¹⁷⁰。彼は Turbo Pascal や Delphi を手がけ C# および .NET Framework の設計者でもあり、プログラミング言語業界におけるカリスマ的存在だった」

「へー、そんなにすごい人が関わってたんですね」

「彼が社内にはいたのは願ってもない幸運だったろうね。相談を受けた Hejlsberg は、JavaScript は全面改修しなければならないほどブロークンな言語だとは考えなかった。そこで JavaScript にすでにあるものはそのままに、大規模開発に必要と思われた静的型付けとクラスベースオブジェクト指向をつけ加えたスーパーセットとして、コードネーム『Strada』の言語がデザインされた。そして 2010 年には

¹⁶⁸ このあたりの経緯は「The Real Story Behind ECMAScript 4」に詳しい。

<https://auth0.com/blog/the-real-story-behind-es4/>

¹⁶⁹ 以降の逸話は「TypeScript Origins: The Documentary」に詳しい。

<https://www.youtube.com/watch?v=U6s2pdxebSo>

¹⁷⁰ 「アンダース・ヘルスバーグ - Wikipedia」<https://w.wiki/Bxqv>

最初のコンパイラが完成したの。

その後 Strada は TypeScript に改名されたけど、その時点でもまだ TypeScript はあくまで社内開発のためのツールだった。しかし開発を進めるにつれその出来に自信を深めたチームは、オープンソースとして公開して広く社外にも使ってもらいたいと考えるようになる。そしてコペンハーゲンで開催された GOTO Conference 2012 のキーノートで Anders Hejlsberg が TypeScript を紹介し¹⁷¹、それに合わせてバージョン 0.8 が外部に公開された」

「ふむふむ、当時の反応はどうだったんでしょうか？」

「おおむね好評だったみたいだね。発表では TypeScript Playground を使って参照元の表示やサジェストなどのコード支援機能がいっしょに実演されたので、そのインパクトも大きかった。ちなみに作りかけだった IDE は Eclipse の設計リーダーで Visual Studio 開発の中心的存在だった Erich Gamma のチームに引き継がれ、Visual Studio Code として 2015 年に公開された。VS Code は当初から TypeScript で開発され、その知見が随時フィードバックされるという、流行りのドッグフーディング¹⁷²で TypeScript は改良が重ねられてきたわけ」

「へーえ、そんな経緯があったんですね。でも Microsoft がオープンソースで成功したってのは意外ですね。一昔前はオープンソースを敵視してた悪の帝國的なイメージがありましたけど」

「数々の好条件が重なった奇跡みたいなものだったかもね。TypeScript はその後、Microsoft の OSS プロジェクトのモデルケースになった。GitHub の買収話が持ち上がったとき、他社を退けて MS が買収できたのも、TypeScript を筆頭とした OSS への実績が影響したと言われてる。また JavaScript に静的型付けを追加する試みには Google と Meta も参入したわけだけど、開発者コミュニティを味方につけるという点で Microsoft にかなり遅れを取ってしまった」

「えっ、Google と Facebook も TypeScript みたいなを出してたんですか？」

「実は Google は TypeScript より 1 年前に同じ GOTO Conference で Dart¹⁷³ を公開してた。Dart は Java や C# に強く影響を受けた言語で、JavaScript の置き換えを狙った完全な別言語。その思想は JavaScript の可能性を信じた Hejlsberg と対症的だね。Google には Chrome に VM を載せる計画まであったんだけど、コミュニティに受け入れられずその試みは頓挫してしまった。そして一時期は『もっとも学ぶ価値のない言語ワースト 1 位¹⁷⁴』という不名誉な称号を得るまでになったんだけど、

¹⁷¹ 「TypeScript Keynote • Anders Hejlsberg • GOTO 2012 - YouTube」
https://www.youtube.com/watch?v=3dqZW_DqHIQ

¹⁷² 自社で開発している製品・サービスを社内で日常的に使用しながら、製品テストを行い改良に役立てること。

¹⁷³ <https://dart.dev/>

¹⁷⁴ 「Worst Programming Languages to Learn in 2018」
<https://www.codementor.io/blog/worst-languages-to-learn-3phycr98zk>

Flutter¹⁷⁵ の開発言語に採用されて復活を果たしたのは周知のとおり」

「へー、Dart って最初は JS の置き換えを狙って作られたんですか。最近は Flutter が人気ですけど、Dart にそんな苦難の歴史があったんですね……」

「また Facebook は JavaScript の静的型チェッカーである Flow を 2014 年に OSS として公開した。強力な型推論や厳密な Null 安全性など、当初は TypeScript より優れている点も少なくなく、React 開発者を中心に採用する人も多かった。私も React を始めた当時、TypeScript を使うか Flow を使うか悩んだよ」

「でも今は Flow って全然聞きませんよね。なんで TypeScript に負けちゃったんでしょうか？」

「あくまで社内向けツールのつもりだったのか、TypeScript みたいに開発者のニーズをうまく汲み上げて、コミュニティといっしょに発展していこうという姿勢が見られなかったんだよね。また TypeScript は TypeScript で作られていて開発に参加しやすかったっぽう、Flow は OCaml なんてマイナー言語で作られていたので敷居がめっちゃ高かった。

だから Flow は機能的に優っていたところも、すぐに TypeScript に追いつき追い越されてしまった。他にもサードパーティのライブラリ向けの型ファイルが少ない、エラーメッセージがわかりにくいといった不満点もあり、2020 年前後には Flow を使っていた人たちが一斉に TypeScript に乗り換えるケースが相次いだ。Babel なんかもそのクチだね。今ではもう、Meta 社製のプロダクトくらいしか Flow は使われてないんじゃないかな。State of JavaScript 2022 で普段使っている JavaScript コンパイラ型言語を尋ねた結果は次のようになっている¹⁷⁶」

¹⁷⁵ Google が提供しているオープンソースのアプリケーション開発フレームワーク。Dart による単一のコードベースで Android、iOS、Windows、Mac など各種プラットフォーム向けの GUI アプリが開発できる。
<https://flutter.dev/>

¹⁷⁶ https://2022.stateofjs.com/en-US/other-tools/#javascript_flavors



図 8: 普段使っている JavaScript コンパイル型言語 (State of JavaScript 2022 より)

「ほぼ 99%！？ 圧倒的じゃないか、我が軍は！！」

「また Microsoft は TypeScript チームからメンバーを TC39 に参加させているので、ECMAScript との連携も密なの。だから次の ECMAScript に入りそうな機能は、だいたい TypeScript で先取りして実装される。だからこの先も TypeScript と JavaScript の親和性は確保されていると言っていいだろうね」

「それは心強いですね……。ところで React + TypeScript の採用例って有名どころではどんなのがあるんですか？」

「私たちが仕事で使ってる Slack のデスクトップアプリは Electron¹⁷⁷ を用いて React で作られてるけど¹⁷⁸、2017 年 4 月に開発言語を JavaScript から TypeScript に移行したって Slack 社のエンジニアが書いたブログ記事が話題になってたね¹⁷⁹。Slack は表から見る限り Web インターフェースも React 製だし、妥当な線で考えればこれも TypeScript 化されてるんじゃないかな。

他には Airbnb も Web が React 製なのは有名だけど、TypeScript を社内の標準言語にして 2019 年の

¹⁷⁷ GitHub が開発している、Chromium や Node.js などの Web 技術によってクロスプラットフォームのデスクトップアプリケーションを作成できるフレームワーク。ちなみに Visual Studio Code も Electron で作られている。<https://www.electronjs.org/>

¹⁷⁸ 「Slack が高速化されたウェブとデスクトップクライアントを公開 | TechCrunch Japan」
<https://jp.techcrunch.com/2019/07/24/2019-07-22-slack-speeds-up-its-web-and-desktop-client/>

¹⁷⁹ 「Slack での JavaScript から TypeScript への移行 - InfoQ」
<https://www.infoq.com/jp/news/2017/04/going-typescript-slack/>

2月の時点でコードベースの半分以上を JavaScript から移行したって発表してた¹⁸⁰」

「えっ。毎日、社内チャットで使ってる Slack も、個人的に旅行の宿探しによく使う Airbnb も React と TypeScript で作られてるんですか」

「海外大手だけじゃないよ。日本でもリクルートがスタディサブリに¹⁸¹、LINE が LINE NEWS¹⁸²に、Sansan が個人向け名刺アプリ Eight の開発¹⁸³に TypeScript を導入してる事例が公表されてる。また内部のエンジニアから聞く話によると、Yahoo! JAPAN やメルカリ、マネーフォワード、サイボウズといった会社も今では Web の部分を React と TypeScript で開発してるらしいよ」

「国内でも有名企業がそんなに！」

「TypeScript はそもそも大規模なアプリケーション開発のために設計された言語だから、大手ほど導入に積極的になるのは必然なのかもね。導入コストはたしかに低くはないけど、企業レベルの Web フロントエンドのプロダクトを継続的に開発していくなら、現在においては必須の技術と言えるんじゃないかな」

「なんだか目からウロコが落ちた思いです。俄然、TypeScript やる気になってきました！」

「いいねー。じゃ、次からは実際にコードを書きながら TypeScript を学んでいこう」

4-2. TypeScript の基本的な型

4-2-1. 型アノテーションと型推論

「TypeScript のコードは typescript パッケージに含まれる tsc というコンパイラを経由することで実行可能な JavaScript ファイルに変換される。でも学習のためには TypeScript を直接実行できる環境があると便利。たとえばブラウザで手軽に試してみたいときは TypeScript Playground¹⁸⁴ という公式に用意されたサイトが使える」

「へー、こんなサイトがあるんですね」

「今回はローカルに実行環境を作りたい。default-npm-packages ファイルによってすでに tsx¹⁸⁵ とい

¹⁸⁰ 「Airbnb における GraphQL マイグレーション - InfoQ」
<https://www.infoq.com/jp/news/2020/02/airbnb-graphql-migration/>

¹⁸¹ 「スタディサブリにおける TypeScript 活用事例と今後の展望 / jsconf-jp-2021 - Speaker Deck」
<https://speakerdeck.com/indigolain/jsconf-jp-2021>

¹⁸² 「120 億 PV の巨大サービス「LINE NEWS」を TypeScript 化した話 - ログミーTech」
<https://logmi.jp/tech/articles/322702>

¹⁸³ 「TypeScript の導入で開発現場はどう変わる？ Sansan の事例に見るメリットとコスト：CodeZine」
<https://codezine.jp/article/detail/14849>

¹⁸⁴ <https://www.typescriptlang.org/play>

¹⁸⁵ <https://tsx.is/>

う実行ツールがインストールされているはずだけど、これは型チェックをしてくれないので REPL で動かしながら学ぶという用途に適してない。なので、少し古い環境にしか対応してないんだけど REPL には `ts-node`¹⁸⁶ を使うことにしよう。ts-node は v20 以降の Node.js ではうまく動かないので、v18 を入れておく」

```
$ mise use -g node@18
$ npm install -g ts-node
```

「typescript パッケージは npm の設定ファイル `.default-npm-packages` に記載していてすでにグローバルインストール済みのはずだから¹⁸⁷、コマンド `tsc -v` で確認してみて。もしなければ `npm install -g typescript` であらためてインストールしておこう。それから `ts-node` を REPL として起動するには CommonJS モードにする必要があるんだけど、そのためには実行ディレクトリに次のような設定ファイル `tsconfig.json` を用意しておくこと」

リスト 67: tsconfig.json

```
{
  "compilerOptions": {
    "target": "es2022",
    "module": "commonjs",
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true
  }
}
```

「`tsc --init` を実行することでもこれとほぼ同じファイルが作られる。なおこの環境では `.ts` ファイルをモジュールとして実行できないので、既存のファイルからコードの動作確認をしたい場合は以下の方法を使ってね」

- `ts-node` で REPL を起動、`.load <ファイル名>` でファイルを読み込む（※ Node.js は v18 を使用のこと）
- `tsc --noEmit --strict <ファイル名>` で結果をファイルに出力せず、コンパイルエラーのみをチェック

¹⁸⁶ <https://typestrong.org/ts-node/>

¹⁸⁷ 「1-1-2. Node.js をインストールする」を参照のこと。

- `tsx <ファイル名>` で型チェックなしに実行

「もろもろ用意できたら `ts-node` コマンドを単体で実行してみよう。REPL が立ち上がるので、これで色々と試してみるよ」

```
> let n: number = 3;
> n = 'foo';
<repl>.ts:5:1 - error TS2322: Type 'string' is not assignable to type 'number'.
```

「最初の行を見るとわかるように、TypeScript では変数の宣言時に **値: 型** というフォーマットで型の注釈がつけられるのね。これを**型アノテーション** (Type Annotation) と呼ぶ。型アノテーションによって静的に型付けされた情報はコンパイル時のチェックに用いられ、書かれたコード中に型の不整合があるとコンパイルエラーになる」

「だから数値型 `number` として型宣言された `n` には文字列型のデータが代入できず、エラーになってるんですね」

「そのとおり。じゃ、これを踏まえて JavaScript との挙動のちがいを見てみよう」

```
$ node
> const s = '123';
> const n = 456;
> s * 3
369
> 246 / s
2
> s + n
'123456'

$ ts-node
> const s = '123';
> const n = 456;
> s * 3
<repl>.ts:6:1 - error TS2362: The left-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type.
> 246 / s
<repl>.ts:6:7 - error TS2363: The right-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type.
> s + n
'123456'
```

「JavaScript には悪名高い**暗黙の型変換**¹⁸⁸という機能があって、この例のように文字列データを算術演算しようとする、勝手にその変数の型を数値型に変換してしまう。いっぽう TypeScript では、数値型と互換性のない型のデータは算術演算が許されないのでコンパイルエラーになる」

「あれ？ でも + はエラーにならず処理されてますね」

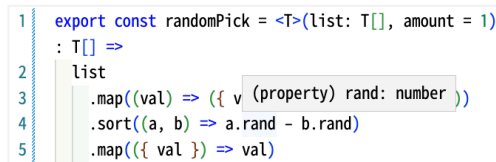
「文字列の + は加算ではなく連結として評価されるから、致命的な不具合は起きないと判断して残したんだろうね。でも今度は右辺の 456 が文字列に変換されてる。ただまあ、この挙動は Scala や Swift でも同じだから、そうおかしいものじゃないのかも」

「そうなんです」

「ところでこのサンプルの TypeScript のほう、さっき説明したばかりの型アノテーションを省略してただけで気づいてた？」

「あ、そういえば s にも n にもないですね。これでも動くんだ」

「処理系がその文脈からその型を推測できる場合は、型アノテーションを省略しても自動的に補完して解釈してくれるの。これを**型推論** (Type Inference) という。ちなみにその変数がコンパイラにどう型推論されてるかというのは、VS Code ならその上にマウスカーソルをホバーさせると、ポップアップでその型を教えてくれるよ。ほら」



```

1 export const randomPick = <T>(list: T[], amount = 1)
2   : T[] =>
3     list
4     .map((val) => ({ v (property) rand: number }))
5     .sort((a, b) => a.rand - b.rand)
6     .map(({ val }) => val)

```

図 9: Visual Studio Code での型のホバーヒント

「おおー、これはかなり便利ですね！」

4-2-2. JavaScript と共通のデータ型

「先に挙動のほうを見ちゃったけど、TypeScript のデータ型についてあらためて基本から説明しておこう。TypeScript は大規模なアプリケーション開発のために作られた、JavaScript に静的型付けの型システムとクラスベースのオブジェクト指向を加えた上位互換の言語。クラスについては歴史的経緯によって例外なんだけど、JavaScript と TypeScript のちがいはほぼその型システムだけだといえる」

「ふむふむ」

¹⁸⁸ 「Type coercion (型強制) | MDN」https://developer.mozilla.org/ja/docs/Glossary/Type_coercion

「JavaScript は変数への型付けが動的なだけで、データそのものの型は存在してるって話はすでにしたよね¹⁸⁹。TypeScript の型システムはこれを抱合する形になってる。よってプリミティブ型は JavaScript と共通した次の7種類になる」

- **Boolean 型** `true` および `false` の2つの真偽値を扱うデータ型。型名は `boolean`
- **Number 型** 数値を扱うためのデータ型。型名は `number`
- **BigInt 型** `number` 型では表現できない大きな数値 (2^{53} 以上) を扱う型。型名は `bigint`
- **String 型** 文字列を扱うためのデータ型。型名は `string`
- **Symbol 型** 「シンボル値」という固有の識別子を表現する値の型。型名は `symbol`
- **Null 型** 何のデータも含まれない状態を明示的に表す値。型名は `null`
- **Undefined 型** 「未定義」であることを表す値。型名は `undefined`

「なるほど、プリミティブ型は JavaScript と全く同じなんですね」

「挙動についてもさっきの説明に付け加えることは特にないかな。じゃあ次は配列の型を見てみよう」

```
const numArr: number[] = [1, 2, 3];
```

「型名の後ろに `[]` をつけると、その型データの配列になるんですね。直感的でわかりやすいです」

「なお配列は `Array` オブジェクトとして定義する別の書き方もできるよ」

```
const strArr: Array<string> = ['one', 'two', 'three'];
```

「あ、これって Java のジェネリクスと同じですよ」

「うん、TypeScript でも同じく **ジェネリクス** (Generics) ¹⁹⁰ と呼ぶ。前者と後者の書き方、どちらでも挙動は同じなんだけど、一般的なコーディングルールでは後者よりも前者のスタイルが推奨されることが多いので、`[]` で配列の型を表現しましょう」

「わかりました。そっちのほうが直感的ですね」

¹⁸⁹ 「2-3. JavaScript のデータ型」を参照のこと。

¹⁹⁰ 「総称型」とも。静的型付けシステムのあるプログラミング言語において、データ型の定義とそれを参照する式の一部にパラメータを渡すことで、類似した構造を持つ複数のデータ型を一括定義できるしくみのこと。

「続いてはオブジェクトの型定義について。これには `object` という型名があるものの、JavaScript のところで説明したように `Object` オブジェクトはプリミティブ型以外のすべてのオブジェクトのプロトタイプになってて、型注釈として使うには広義すぎて用をなさないため、ほとんど使われない。たとえばこれが通ってしまうわけだから」

```
const words: object = ['foo', 'bar', 'baz'];
```

「なるほど、たしかにこれは無意味ですね」

「だから狭義のオブジェクトの型を定義する際は、プロパティのキー名と値の型を明記する形でこのように型アノテーションを行うの」

```
const red: { rgb: string, opacity: number } = { rgb: 'ff0000', opacity: 1 };
```

「……ふむ、でもこれ毎回インラインで書くの大変じゃないですか？」

「たしかにそうだね。だから TypeScript ではオブジェクトの型に名前をつけることができるようになってる。それが **インターフェース** (Interface) と呼ばれるもの。こういうふうに定義して使う」

リスト 68: 02-basic-types/interface-sample.ts

```
interface Color {
  readonly rgb: string;
  opacity: number;
  alias?: string;
}

const tq: Color = { rgb: '00afcc', opacity: 1 };
tq.alias = 'turquoise';
tq.rgb = '03c1ff'; // error TS2540: Cannot assign to 'rgb' because it is a read-only property.
```

「説明のついでに、プロパティに特殊な属性をつける記法を入れておいた。まず `readonly` 修飾子をつけたプロパティは書き換え不可になる。それからプロパティ名の末尾に `?` をつけると、そのプロパティは省略可能になる」

「ふむふむ。だから `tq` オブジェクトは `alias` プロパティがなくても作成できたし、後から `rgb` プロパティに値を代入しようとするとながら怒られたんですね」

「そう。さらに、プロパティをもっと柔軟に定義する記法もあるよ」

リスト 69: 02-basic-types/index-signature.ts

```
interface Status {
  [parameter: string]: number;
```

```
}  
  
const myStatus: Status = {  
  level: 22,  
  experience: 3058,  
  maxHP: 156,  
  maxMP: 174,  
  attack: 39,  
  defense: 25,  
};
```

「このインターフェース `Status` はキーが文字列で値が数値というプロパティを持つオブジェクトの型を定義してる。`[key_name: key_type]` という記法でキーの型を縛ってるわけだけど、これを**インデックスシグネチャ** (Index Signature) という。シグネチャとはコンピュータ用語で要素を識別するための情報という意味で用いられるので、キー (インデックス) の型を一意に特定するということね。

気をつけないといけないのは、インデックスシグネチャを使うとすべてのプロパティがその定義に従う必要があること。だから `Status` インターフェースを用いてオブジェクトに `job: 'maze'` なんてプロパティは定義できない」

「ほんとだ。やってみたら `Type 'string' is not assignable to type 'number'` って型エラーになりますね」

「インデックスシグネチャを用いてキーの文字列を任意のパターンから限定したり、値の型の種類にもっと幅を持たせることもできるんだけど、それは他の機能と組み合わせる必要があるので後でもう一度説明するね」

4-2-3. Enum 型とリテラル型

「HTML のセレクトボックスのように列挙したものの中からひとつだけ限定したい場合に TypeScript ではどういう型が使えるか。2 つ方法があるんだけど、まず挙げられるのが **Enum 型**¹⁹¹」

「`enum` って Java にも Ruby にもありますよね。言葉の意味もあまりピンとこないし、あまり使ってなかったですけど」

「Enum は enumeration (列挙) の略で、日本語でも直訳で『列挙型』とされる。いま言及してくれたように他の言語にも `enum` はあるけど、TypeScript の `enum` はちょっと挙動に問題がある」

¹⁹¹ 「TypeScript: Handbook - Enums」

<https://www.typescriptlang.org/docs/handbook/enums.html>

第2部「React 基礎編」へつづく

著者紹介

大岡由佳（おおおか・ゆか）

国内の IT 企業数社での PHP や Ruby による開発およびプロダクトマネージャを経て、2016 年からフリーランスに転身。当時まだ日本ではマイナーな存在だった React に惚れ込み、React 専門のフリーランスエンジニアとして複数の現場を渡り歩く。その経験を元に執筆した「りあクト！」シリーズが評判を呼びヒット作となる。現在は事実上おそらく国内（世界でも？）に唯一生息する、専門の技術同人作家。

趣味は 3.5 倍速で聴くオーディオブックの読書と、ジム代わりに 7 年続いているバレエ。

X アカウントは @oukayuka。ブログは <https://klemiwary.com>（くるみ割り書房公式サイト）。

イラスト：黒木めぐみ（くろき・めぐみ）

漫画家、イラストレーター。

「りあクト！」シリーズの表紙イラストを一貫して担当。

りあクト！ TypeScript で始めるつらくない React 開発 第 5 版

【② React 基礎編】



現場のエンジニアから多大な支持を受ける『りあクト！ TypeScript で始めるつらくない React 開発』の最新 5 版、全 3 部構成の第 2 部「React 基礎編」。

本質を理解するため歴史を深堀りして、なぜ React が今のようになったかを解き明かします。そのうえで JSX やコンポーネント、Hooks を学ぶため、理解度が格段にちがってくるはず。初～中級者の方にもオススメです。BOOTH にて絶賛販売中！

(2025 年 5 月 30 日発行／B5 判・216p／電子版 ¥1,200／紙＋電子セット ¥1,550)

【目次】

- 第 5 章 JSX で UI を表現する
- 第 6 章 React をめぐるモダン Web 開発の歴史（前編）
- 第 7 章 ビルドツールでプロジェクトを運用する
- 第 8 章 コンポーネントの基本を学ぶ
- 第 9 章 リンターとフォーマッターでコード美人に
- 第 10 章 React をめぐるモダン Web 開発の歴史（後編）

りあクト！ TypeScript で始めるつらくない React 開発 第 5 版

【③ React 実践編】



現場のエンジニアから多大な支持を受ける『りあクト！ TypeScript で始めるつらくない React 開発』の最新 5 版、全 3 部構成の第 3 部「React 実践編」。

19 ではサーバ稼働が前提の機能が追加され、もはや React はフレームワークで使われるのが当然の時代。そこで第 5 版では SPA に加えて Nex.js および React Router（旧 Remix）での各種機能の使い方を丁寧に紹介しています。

React 未経験者にはもちろん、19 の新機能やフレームワークについて学びたい初～中級者の方にもオススメです。BOOTH にて絶賛販売中！

（2025 年 5 月 30 日発行／B5 判・215p／電子版 ¥1,200／紙＋電子セット ¥1,550）

【目次】

- 第 11 章 React のページをルーティングする
- 第 12 章 外部リソースのデータを取得する
- 第 13 章 エラーをハンドリングする
- 第 14 章 メモ化でレンダリングを効率化する
- 第 15 章 フォームでデータを更新する

メールマガジン登録のご案内



くるみ割り書房 BOOTH ページ

「りあクト！」シリーズを刊行している「くるみ割り書房」では、新刊や紙の本の再版予定、その他読者の方へのお知らせなどをメールマガジン形式で不定期に配信しています。

購読をご希望の方は、「くるみ割り書房 BOOTH」で検索して最初に表示された上記画像のページにて、「フォロー」ボタンからサークルのフォロー登録をお願いします。BOOTH の一斉送信メッセージにてメルマガの内容をお届けします。

(※ BOOTH のアカウントが必要になります。BOOTH はピクシブ株式会社が運営するオンラインショップです)

りあクト！ TypeScript で始めるつらくない React 開発 第5版

【① 言語・環境編】

2025 年 5 月 30 日	初版第 1 刷発行
2025 年 5 月 30 日	電子版バージョン 1.0.0

著	者	大岡 由佳
発	行	元
連	絡	先
		くるみ割り書房
		oukayuka@gmail.com

本書を無断で複写・複製、転載、データ配信、オークションやフリマアプリに出品することを固く禁じます。

© 2025 Yuka O'oka / Klemiuary Books